N057 Expo로 휴대폰 앱 만들기_해설서

Chapter01

챕터 01. Expo로 휴대폰 앱 만들기

핵심 주제: 웹 서비스를 모바일 앱으로 확장하기

학습 단계: AI 에이전트 서비스 실전 개발

1. 강의의 목적

이번 강의의 핵심은 이미 웹으로 만들어 놓은 AI 친구 에이전트를 스마트폰에서도 사용할 수 있도록 모바일 앱으로 확장하는 방법을 배우는 것입니다.

지금까지 만든 서비스는 웹 주소를 입력해 접속해야 합니다. 하지만 실제 서비스라면 스마트폰 홈 화면의 아이콘을 누르고 바로 실행할 수 있어야 합니다.

이를 위해 Swift나 Kotlin을 처음부터 배우는 대신, React Native와 Expo를 활용해 Windows 환경에서도 모바일 앱을 개발하는 방법을 익힙니다.

특히 기존 백엔드, 데이터베이스, 로그인 시스템을 최대한 재사용하면서 모바일 화면과 스마트폰 기능을 추가하는 것이 중요합니다.

. 중학생도 이해하는 핵심 개념

① 웹 서비스와 모바일 앱의 차이

웹 서비스와 모바일 앱은 사용자가 서비스를 이용하는 출입구가 다릅니다.

- 웹 서비스: 브라우저를 열고 인터넷 주소로 접속합니다.
- 모바일 앱: 스마트폰에 설치된 아이콘을 눌러 실행합니다.

예를 들어 같은 AI 친구와 대화하더라도 웹에서는 크롬 브라우저를 통해 접속하고, 앱에서는 스마트폰에 설치된 AI 친구 아이콘을 눌러 접속합니다.

같은 AI 서비스, 서로 다른 접속 방식

Direct Message | Daily UI Design by Ecem Karademir Güven on Dribbble

웹 서비스

브라우저 → 웹 주소 → 서버

تعرف على تطبيقات المراسلة التي لا تقرأ رسائلك

모바일 앱

앱 아이콘 → 앱 실행 → 서버

공통점: 두 방식 모두 같은 백엔드와 데이터베이스를 사용할 수 있습니다.

② 플랫폼, 네이티브, 빌드란?

용어	쉬운 설명	예시
플랫폼	프로그램이 실행되는 환경	iOS, Android, Web
네이티브	해당 운영체제에 맞춰 만든 앱	Swift 기반 iOS 앱
빌드	개발한 코드를 실행 가능한 앱으로 만드는 과정	APK, AAB, IPA
백엔드	서비스의 핵심 업무를 처리하는 서버	AI 친구 응답 처리
프런트엔드	사용자가 직접 보는 화면	채팅창, 버튼, 메뉴

비유하면 다음과 같습니다.

- 플랫폼은 자동차가 달리는 도로입니다.
- 네이티브 앱은 도로에 맞춰 특별히 제작한 자동차입니다.
- 빌드는 자동차의 설계도를 실제 자동차로 만드는 과정입니다.

③ 모바일 앱을 만드는 네 가지 방법

이번 강의에서는 모바일 앱을 만드는 방법을 비교하고 Expo를 선택합니다.

방식	특징	주요 한계
PWA	웹사이트를 앱처럼 설치	기기 기능 및 OS별 지원에 제약
웹 래핑	기존 웹사이트를 앱 껍데기로 감쌈	네이티브 기능 활용에 제약
네이티브	iOS와 Android에 맞춰 각각 개발	개발 비용과 관리 부담이 큼
React Native + Expo	JavaScript 기반으로 모바일 앱 개발	일부 네이티브 기능은 별도 설정 필요

핵심은 React Native와 Expo입니다.

React Native는 JavaScript와 React를 활용해 모바일 앱 화면을 만드는 기술이고, Expo는 React Native 앱을 쉽게 개발하고 테스트하고 빌드하도록 도와주는 도구 모음입니다.

A Guide to Expo App Development for Modern Teams - Nerdify Blog

getnerdify.com

④ Expo가 해주는 일

Expo는 모바일 앱 개발에 필요한 여러 복잡한 작업을 대신 처리해 줍니다.

구분	Expo의 역할
개발	앱 화면과 기능 개발 지원
실행	Expo Go를 이용해 실제 스마트폰에서 테스트
빌드	앱 설치 파일 생성 지원
배포	앱스토어 출시를 위한 빌드와 배포 과정 지원

여기서 알아야 할 중요한 차이가 있습니다.

- Expo Go: 개발 중인 앱을 빠르게 스마트폰에서 실행해 보는 테스트용 앱입니다.
- 개발용 빌드: 프로젝트에 필요한 네이티브 기능과 설정을 포함해 별도로 만든 개발용 앱입니다.

Expo Go는 모든 네이티브 기능을 지원하는 것은 아니므로, 실제 서비스 개발에서는 개발용 빌드가 필요할 수 있습니다.

또한 Windows에서도 앱 개발은 가능하지만, iOS 앱의 최종 빌드와 배포에는 Expo의 클라우드 빌드 서비스를 활용할 수 있습니다.

3. 기존 웹 서비스와 모바일 앱의 연결 구조

이번 강의에서 가장 중요한 부분입니다.

모바일 앱을 만든다고 해서 기존 AI 서버와 데이터베이스를 다시 만들 필요는 없습니다.

기존 웹 서비스의 핵심 기능을 그대로 활용하고, 모바일 화면을 새롭게 만드는 것입니다.

기존 웹 서비스와 모바일 앱의 구조

사용자 접속 화면

웹 브라우저

모바일 앱

공통 백엔드 서버

로그인 · 설정 조회 · AI 에이전트 · 응답 스트리밍

공통 데이터베이스

사용자 정보 · 대화 기록 · 서비스 설정

이를 통해 웹과 모바일 앱이 동일한 사용자 정보와 대화 기록을 공유할 수 있습니다.

예를 들어 PC에서 AI 친구와 나눈 대화를 스마트폰에서도 이어서 볼 수 있는 것입니다.

앱에서 AI 답변을 받는 과정

1. 앱을 실행하고 로그인합니다.
2. 백엔드에서 사용자 설정을 가져옵니다.
3. 사용자가 메시지를 입력합니다.
4. 앱이 메시지를 서버에 전달합니다.
5. 서버가 AI 모델에 요청합니다.
6. AI 응답이 생성되는 대로 앱에 스트리밍 방식으로 전달됩니다.
7. 앱 화면에 답변이 실시간으로 표시됩니다.

스트리밍은 답변 전체가 완성될 때까지 기다리는 것이 아니라, 글자가 생성되는 대로 조금씩 보여주는 방식입니다.

4. 실제 활용: 카메라로 찍은 일정표를 약속으로 만들기

이번 강의에서 특히 주목할 부분은 스마트폰의 카메라와 사진첩을 활용하는 기능입니다.

Schichtplan zu Kalender: Dienstpläne und Schichten digitalisieren | Smart Calendars AI | Smart Calendars AI

Google Photos cloud content will soon appear in Android's Photo Picker, finally

From a handout straight to the calendar! "Event Scan Function" for Android - TimeTree

8

예를 들어 종이에 적힌 일정표를 사진으로 찍으면 다음과 같은 흐름으로 처리할 수 있습니다.

단계	사용하는 기술	역할
1	expo-image-picker	사진첩에서 이미지 선택
2	expo-camera	앱 내부에서 사진 촬영
3	백엔드 OCR	사진 속 글자 인식
4	AI 분석	날짜, 시간, 장소 추출
5	데이터베이스	약속 정보 저장

여기서 중요한 점은 카메라 기능과 OCR 기능을 구분하는 것입니다.

- 카메라는 사진을 확보하는 역할입니다.
- OCR은 사진 속 문자를 읽는 역할입니다.

- AI는 읽어낸 정보를 이해하고 일정 형태로 정리하는 역할입니다.

이런 기능이 결합되면 단순한 대화형 AI를 넘어 실제 생활을 도와주는 AI 에이전트로 발전할 수 있습니다.

5. 왜 AI 개발자가 알아야 하는가?

이번 강의는 단순히 모바일 앱 화면을 만드는 기술을 배우는 과정이 아닙니다.

AI 서비스를 실제 사용자에게 제공하기 위한 서비스 확장 방법을 배우는 과정입니다.

특히 다음 세 가지가 중요합니다.

첫째, 서비스 접근성입니다.

사용자가 매번 웹 주소를 입력하지 않고 앱 아이콘을 눌러 서비스를 이용할 수 있습니다.

둘째, 스마트폰의 하드웨어 활용입니다.

카메라, 사진첩 등 스마트폰의 기능을 AI와 연결하면 웹만으로 구현하기 어려웠던 새로운 서비스를 만들 수 있습니다.

셋째, 기존 시스템의 재사용입니다.

기존 백엔드와 데이터베이스를 유지하면서 새로운 사용자 접점을 추가할 수 있습니다. 이는 개발 비용과 운영 복잡성을 줄이는 데 도움이 됩니다.

6. 나의 프로젝트와 연결하기

이번 강의에서 배운 내용은 현재 구상하고 있는 AI 서비스들에도 직접 적용할 수 있습니다.

프로젝트	모바일 앱 적용 아이디어
AI 친구 에이전트	매일 대화하고 알림을 받는 개인 AI 친구
MSM(My-Story-Maker)	스마트폰으로 일기, 사진, 음성을 입력해 자서전 제작
Travel Creator	여행 중 사진 촬영, 일정 입력, 여행 기록 자동화
ASP AI 컨설팅	고객이 스마트폰으로 자료를 촬영하고 AI 진단 요청
AI 학습 도우미	강의 자료를 촬영해 요약하고 학습 기록 저장

특히 MSM과 Travel Creator는 모바일 앱의 활용 가치가 큽니다.

MSM은 사용자가 일상에서 순간적으로 떠오르는 생각이나 사진을 쉽게 입력할 수 있어야 합니다. Travel Creator는 여행 중 사진과 위치, 일정 등을 바로 기록할 수 있어야 합니다.

즉, 모바일은 단순히 웹사이트를 작은 화면으로 옮기는 것이 아니라 AI 서비스가 사용자의 일상 속으로 들어가는 중요한 접점입니다.

7. 반드시 기억해야 할 핵심 내용

핵심 항목	기억할 내용
모바일 앱 개발	웹과 별도로 사용자 화면을 개발
React Native	JavaScript 기반 모바일 앱 개발 기술
Expo	React Native 앱의 개발·실행·빌드 지원
Expo Go	스마트폰에서 개발 중인 앱을 테스트
개발용 빌드	네이티브 기능과 설정을 포함한 개발 앱
기존 백엔드	웹과 모바일이 공동으로 사용 가능
스트리밍	AI 답변을 생성되는 대로 전달
expo-image-picker	사진첩에서 이미지 선택
expo-camera	앱 내부에서 카메라 실행
OCR	이미지에서 문자를 추출

8. 오늘 반드시 기억할 5가지

1. 모바일 앱을 만든다고 기존 백엔드와 데이터베이스를 새로 만들 필요는 없습니다.
2. React Native와 Expo를 이용하면 Windows에서도 모바일 앱 개발을 시작할 수 있습니다.
3. Expo Go는 빠른 테스트용이며, 실제 네이티브 기능 개발에는 개발용 빌드가 필요할 수 있습니다.
4. 카메라와 사진첩을 AI 백엔드와 연결하면 현실 세계의 정보를 디지털 데이터로 바꿀 수 있습니다.
5. 모바일 앱 개발의 최종 목표는 앱 파일을 만드는 것이 아니라 사용자가 일상에서 AI 서비스를 편리하게 이용하도록 만드는 것입니다.

9. 오늘 강의 한 문장 정리

Expo는 기존 AI 웹 서비스의 백엔드를 그대로 활용하면서 React Native로 모바일 앱을 개발하고, 스마트폰의 카메라와 사진첩까지 AI 기능과 연결할 수 있도록 도와주는 개발 플랫폼입니다.

학습 방향 제안

이번 강의는 AI 친구 에이전트를 웹에서 모바일로 확장하는 첫걸음입니다. 우선 Expo로 모바일 앱을 완성하고, 이후 MSM과 Travel Creator에도 동일한 개발 구조를 적용해 보시는 것이 좋겠습니다.

Chapter02

챕터 02. 웹 서비스와 앱의 차이

핵심 주제: 웹과 앱의 실행·배포·운영 구조 이해

학습 단계: 모바일 앱 아키텍처

1. 강의의 목적

이번 강의의 핵심은 웹 서비스와 모바일 앱이 어떻게 다르게 실행되고 배포되는지 이해하고, 기존 웹 서비스의 백엔드를 모바일 앱에서도 재사용할 수 있는 이유를 배우는 것입니다.

가장 중요한 차이는 화면 코드가 사용자에게 전달되는 방식입니다.

- 웹: 사용자가 주소를 열 때마다 서버에서 화면 코드를 받아 실행합니다.
- 앱: 앱 파일을 스마트폰에 설치해 놓고 실행합니다.
- 공통점: 두 방식 모두 대화 기록, 로그인, 크레딧 등의 데이터는 백엔드에서 가져올 수 있습니다.

이 차이를 이해해야 모바일 앱을 출시한 이후에도 기존 웹 서비스와 백엔드를 안정적으로 운영할 수 있습니다.

. 중학생도 이해하는 핵심 개념

① 웹 서비스와 모바일 앱은 무엇이 다른가?

가장 쉬운 비유는 인터넷으로 책을 읽는 것과 책을 직접 구매해서 소장하는 것의 차이입니다.

- 웹 서비스는 매번 인터넷 서점에 접속해 책을 가져오는 방식입니다.
- 모바일 앱은 책을 한 번 구매해서 내 책장에 보관하고 읽는 방식입니다.

웹과 앱의 실행 방식 비교

Web browser loading screen

웹 서비스

1. 주소 입력
2. 서버에서 화면 코드 다운로드
3. 브라우저에서 실행

Apple, iPhone ekranları için arayışa girdi! - ShiftDelete.Net

모바일 앱

1. 앱 설치
2. 휴대폰에 저장된 코드 실행
3. 필요한 데이터만 서버에서 수신

가장 중요한 차이

- 웹은 화면 코드를 서버에서 가져옵니다.
- 앱은 화면 코드가 설치 파일에 들어 있습니다.
- 데이터는 두 방식 모두 백엔드에서 가져올 수 있습니다.

여기서 중요한 것은 앱이라고 해서 인터넷이나 서버가 필요 없는 것은 아니라는 점입니다.

AI 친구의 대화 내용, 사용자 정보, 크레딧은 여전히 백엔드에서 관리합니다.

② 앱을 출시한 다음에는 왜 관리가 더 복잡해질까?

이 부분이 오늘 강의에서 가장 중요합니다.

웹은 서버에 새 화면을 배포하면 사용자가 다음에 접속할 때 새로운 화면을 보게 됩니다.

하지만 앱은 다릅니다.

사용자가 앱을 설치하면 그 당시의 화면 코드가 휴대폰에 남습니다.

예를 들어 다음과 같은 상황을 생각해 보겠습니다.

- A 사용자: 최신 앱으로 업데이트했습니다.
- B 사용자: 앱을 업데이트하지 않았습니다.
- 서버: 새로운 API 응답 형식으로 변경했습니다.

이때 A는 새로운 응답을 이해하지만 B는 과거의 필드 이름을 찾기 때문에 오류가 발생할 수 있습니다.

앱 버전이 달라지면 발생하는 문제

최신 앱

V2

새로운 API 응답 이해 가능

구버전 앱

V1

새로운 API 응답을 이해하지 못할 수 있음

공통 백엔드 API V2

설치된 앱 버전이 서로 다르기 때문에 백엔드가 구버전 앱과의 호환성도 고려해야 합니다.

따라서 앱을 운영할 때는 백엔드 API의 응답 구조를 함부로 변경해서는 안 됩니다.

이를 **하위 호환성(Backward Compatibility)**이라고 합니다.

쉽게 말해 새로운 시스템을 만들더라도 이전 버전의 사용자들이 계속 이용할 수 있도록 배려하는 것입니다.

③ 운영체제와 플랫폼

스마트폰에는 운영체제(OS)가 있습니다.

운영체제는 스마트폰의 두뇌이자 관리자입니다.

화면, 터치, 카메라, 저장 공간, 네트워크 등을 관리하고 앱이 이러한 기능을 이용할 수 있도록 도와줍니다.

대표적인 운영체제는 두 가지입니다.

Best Refurbished Mobile – Like New Condition | Budli | buy.budli.in

iOS

- 기기: iPhone, iPad
- 개발 언어: Swift
- 개발 도구: Xcode
- 앱 파일: IPA
- 스토어: App Store

Google Pixel 7 Cell Phone Review - Consumer Reports

Android

- 기기: 삼성, Google 등
- 개발 언어: Kotlin
- 개발 도구: Android Studio
- 앱 파일: APK, AAB
- 스토어: Google Play

여기서 플랫폼이라는 말은 운영체제뿐 아니라 개발 도구와 앱을 배포하는 스토어까지 포함하는 개념입니다.

두 플랫폼은 개발 환경이 다르기 때문에 원래는 iOS용과 Android용 앱을 각각 개발해야 했습니다.

④ SDK란 무엇인가?

SDK(Software Development Kit)는 소프트웨어 개발 도구 모음입니다.

쉽게 설명하면 자동차를 만드는 데 필요한 설계도, 부품 설명서, 공구를 한곳에 모아 놓은 개발자용 도구 상자입니다.

예를 들어 Android SDK에는 다음과 같은 것들이 들어 있습니다.

- 스마트폰 기능을 사용하기 위한 라이브러리
- 앱을 컴파일하는 데 필요한 도구
- 앱 파일을 만드는 도구
- 앱에 서명하는 도구

iOS는 Xcode 안에 관련 개발 도구가 포함되어 있습니다.

⑤ 네이티브란 무엇인가?

네이티브(Native)는 해당 운영체제가 기본적으로 제공하는 언어와 화면 요소, 기능을 사용하는 것을 의미합니다.

예를 들어 iPhone의 설정 화면을 생각해 보겠습니다.

How to share your Wi-Fi password from Mac to iPhone

macpaw.com

- 설정 화면의 스위치
- 화면을 위아래로 움직이는 스크롤
- 글자를 입력할 때 나타나는 키보드
- 카메라와 위치 정보 기능

이런 기능은 운영체제가 기본적으로 제공하는 기능입니다.

카메라나 위치 정보처럼 개인정보와 관련된 기능은 사용자의 허락을 받아야 합니다.

웹뷰(WebView)와의 차이

웹뷰는 앱 안에 브라우저를 넣어 웹사이트를 보여주는 방식입니다.

구분	네이티브 화면	웹뷰 화면
화면 구성	운영체제의 기본 요소	웹 기술로 구성

구분	네이티브 화면	웹뷰 화면
실행 방식	앱 내부 코드 실행	앱 안의 브라우저에서 실행
느낌	스마트폰 앱에 자연스러움	웹사이트와 유사
기기 기능	운영체제 API 활용	별도 연결이 필요할 수 있음

이번 강의에서 사용하는 React Native는 중요한 특징이 있습니다.

코드는 JavaScript로 작성하지만 실제 화면은 각 운영체제의 네이티브 요소로 그립니다.

따라서 하나의 코드 기반으로 iOS와 Android 앱을 함께 개발할 수 있습니다.

⑥ 빌드(Build)란 무엇인가?

빌드는 개발자가 작성한 코드를 스마트폰에 설치할 수 있는 앱 파일로 만드는 과정입니다.

쉽게 비유하면 요리 레시피를 실제 음식으로 완성하는 과정입니다.

PPT - Android Outlook PowerPoint Presentation, free download - ID:5426982
slideserve.com

빌드는 다음 순서로 진행됩니다.

순서	작업	쉬운 설명
1	코드 가져오기	개발한 소스 코드 준비
2	패키지 설치	필요한 재료 준비
3	번들(Bundle)	JavaScript 코드를 실행에 필요한 파일로 묶기
4	컴파일(Compile)	네이티브 코드를 기계가 실행할 수 있는 형태로 변환
5	서명(Signing)	개발자가 만든 앱이라는 신원 확인
6	앱 파일 생성	설치 가능한 APK 또는 IPA 생성

여기서 세 가지 용어를 꼭 기억하셔야 합니다.

- 번들: JavaScript 코드와 필요한 파일을 실행 가능한 묶음으로 만드는 것
- 컴파일: 사람이 작성한 코드를 기계가 실행할 수 있는 형태로 변환하는 것
- 서명: 앱을 만든 개발자를 확인하고 파일의 무결성을 검증하는 것

⑦ 왜 Windows에서는 iOS 앱을 직접 빌드하기 어려울까?

이것은 이번 프로젝트와 직접 연결되는 내용입니다.

구분	Android	iOS
빌드 도구	Android SDK, Gradle	Xcode

구분	Android	iOS
Windows에서 빌드	가능	공식 로컬 빌드 불가
Mac 필요 여부	불필요	필요
대안	로컬 빌드	클라우드 Mac 빌드

iOS 앱을 만들 때는 Apple이 제공하는 Xcode와 SDK, 서명 도구가 필요합니다.

그런데 Xcode는 Mac에서만 실행됩니다.

따라서 Windows 노트북을 사용하는 개발자는 iOS 빌드를 위해 Mac이나 클라우드 빌드 서비스를 활용해야 합니다.

여기서 Expo의 가치가 나타납니다.

Expo는 이 복잡한 빌드 과정을 클라우드로 옮겨 Windows에서도 iOS 앱 개발과 빌드를 진행할 수 있도록 지원합니다.

⑧ 앱이 실행되는 동안에도 알아야 할 것

앱은 스마트폰에서 항상 실행되는 것이 아닙니다.

사용자가 홈 화면으로 나가거나 다른 앱을 열면 백그라운드 상태가 됩니다.

메모리가 부족하면 운영체제가 백그라운드 앱을 종료할 수도 있습니다.

예를 들어 AI 친구에게 긴 질문을 하고 답변을 받는 중에 다른 앱으로 이동했다고 가정해 보겠습니다.

이때 앱이 종료되면 스트리밍 연결이 끊길 수 있습니다.

그래서 데이터의 저장 위치를 잘 구분해야 합니다.

데이터	저장 위치	이유
로그인 토큰	스마트폰 저장소	앱을 다시 열어도 로그인 유지
대화 기록	백엔드 데이터베이스	다른 기기에서도 확인
크레딧	백엔드 데이터베이스	사용자별 사용량 관리
화면 설정	스마트폰 또는 백엔드	필요에 따라 관리

핵심은 앱이 종료되어도 보존해야 하는 정보는 적절한 저장소에 남겨야 한다는 것입니다.

3. 오늘 배운 내용을 실제 프로젝트에 적용하기

현재 개발 중인 AI 친구 에이전트를 기준으로 생각해 보겠습니다.

웹 서비스에서 모바일 앱으로 확장할 때 다음과 같이 구분할 수 있습니다.

구성 요소	기존 웹 서비스	모바일 앱
사용자 화면	웹 화면	React Native 화면
로그인	기존 로그인 API	기존 로그인 API 재사용 가능
AI 대화	기존 백엔드	기존 백엔드 재사용
대화 기록	기존 데이터베이스	기존 데이터베이스 재사용
크레딧	기존 백엔드 관리	기존 백엔드 관리
앱 실행	브라우저	스마트폰 설치 앱
업데이트	웹 배포	앱 재빌드 및 업데이트
카메라	브라우저 기능	네이티브 기능 연동

따라서 개발 전략은 다음과 같습니다.

1. 기존 백엔드와 데이터베이스는 유지합니다.
2. React Native와 Expo로 모바일 화면을 만듭니다.
3. 기존 로그인 및 AI 대화 API를 연결합니다.
4. 스마트폰에 필요한 카메라와 사진첩 기능을 추가합니다.
5. Android와 iOS 앱을 각각 빌드하고 배포합니다.
6. 구버전 앱도 정상적으로 작동하는지 확인합니다.

4. 왜 AI 서비스 개발자가 알아야 하는가?

이번 강의는 단순한 모바일 개발 지식이 아니라 AI 서비스의 운영 구조를 이해하는 데 필요한 기본 지식입니다.

특히 세 가지 관점이 중요합니다.

첫째, 프런트엔드와 백엔드를 분리해서 생각해야 합니다.

사용자가 보는 화면은 웹에서 모바일로 바뀌어도 AI의 핵심 처리 기능은 기존 백엔드에서 유지할 수 있습니다.

둘째, 앱은 출시 이후에도 버전 관리가 필요합니다.

웹은 배포하면 비교적 즉시 반영되지만, 앱은 사용자마다 설치된 버전이 다를 수 있습니다. 따라서 API 호환성을 고려한 설계가 중요합니다.

셋째, 스마트폰의 운영체제 기능을 이해해야 합니다.

카메라, 사진첩, 위치, 알림 등은 AI 서비스에 새로운 입력과 활용 방식을 제공합니다.

5. 반드시 기억해야 할 핵심 용어

용어	한 줄 설명
OS	스마트폰의 자원을 관리하는 운영체제
플랫폼	운영체제, 개발 도구, 배포 환경을 포함하는 개념
SDK	개발에 필요한 라이브러리와 도구 모음
네이티브	운영체제가 기본 제공하는 기술과 기능
웹뷰	앱 안에서 웹 페이지를 보여주는 브라우저
빌드	소스 코드를 설치 가능한 앱 파일로 만드는 과정
번들	JavaScript 코드와 필요한 파일을 묶는 과정
컴파일	코드를 기계가 실행할 수 있는 형태로 변환
서명	앱 개발자와 파일의 무결성을 확인하는 과정
패키지 이름	앱을 구분하는 고유 식별자
권한	카메라, 위치 등 기능 사용에 필요한 사용자 허락
하위 호환성	기존 버전 사용자도 새 시스템을 이용할 수 있게 하는 특성

6. 객관식 퀴즈 정답 및 해설

정답: 2번

문제: 백엔드 API 응답의 필드 이름을 바꾸고 앱도 새 이름을 읽도록 고쳐 새 버전을 출시하려고 합니다. 웹 화면만 있을 때는 없던 문제는 무엇인가요?

정답은 2. 업데이트하지 않은 사용자의 폰에서는 옛 앱이 옛 필드 이름을 찾으므로, 그 사용자의 화면이 깨진다입니다.

이유는 다음과 같습니다.

- 웹은 서버에 새 화면을 배포하면 다음 접속부터 새 코드가 적용됩니다.
- 앱은 사용자의 스마트폰에 기존 코드가 남아 있습니다.
- 사용자가 업데이트하지 않으면 과거 버전의 코드로 새로운 백엔드에 접속합니다.
- 새로운 API 응답을 이해하지 못하면 오류가 발생할 수 있습니다.

따라서 백엔드를 변경할 때는 기존 앱과의 호환성을 고려해야 합니다.

7. 오늘 강의에서 반드시 기억할 5가지

1. 웹은 서버에서 화면 코드를 받아 실행하지만 앱은 설치된 파일을 실행합니다.
2. 웹과 앱은 사용자 화면만 다를 뿐 백엔드와 데이터베이스를 공유할 수 있습니다.

3. 앱은 업데이트하지 않은 사용자에게 구버전이 남기 때문에 API 하위 호환성이 중요합니다.
4. React Native는 JavaScript로 개발하면서 iOS와 Android의 네이티브 화면을 구현할 수 있습니다.
5. Expo는 Windows 개발자가 iOS 빌드를 진행할 때 필요한 Mac 기반 빌드 환경을 클라우드로 활용할 수 있도록 도와줍니다.

8. 오늘 강의 한 문장 정리

웹과 모바일 앱의 가장 큰 차이는 화면 코드의 실행과 배포 방식이며, 기존 백엔드를 공유하되 앱의 버전 관리와 API 하위 호환성을 고려하는 것이 모바일 서비스 운영의 핵심입니다.

다음 강의로 연결되는 생각

이번 챕터에서 가장 중요한 것은 화면과 백엔드를 분리해서 생각하는 습관입니다.

앞으로 MSM, Travel Creator, AI 친구 에이전트를 개발할 때도 화면은 웹과 앱으로 확장하되, 핵심 비즈니스 로직과 데이터는 백엔드에서 관리하는 구조를 기본 원칙으로 삼으시면 좋겠습니다.

Chapter03

챕터 03. 웹 서비스를 모바일 앱으로 만드는 네 가지 방법

핵심 주제: 모바일 앱 개발 방식 비교와 선택

학습 단계: 기술 선택 및 의사결정

1. 강의의 목적

이번 강의의 핵심은 기존 웹 서비스를 모바일 앱으로 전환하는 네 가지 방법을 이해하고, 내 서비스의 상황에 가장 적합한 기술을 선택하는 것입니다.

웹 서비스를 모바일로 가져가는 방법은 크게 네 가지입니다.

1. PWA
2. 웹 래핑(Web Wrapping)
3. 네이티브(Native)
4. React Native + Expo

네 가지 방법은 화면을 어떻게 그리는지, 앱을 어떻게 설치하는지, 개발 도구가 얼마나 필요한지, 스마트폰 기능을 얼마나 활용할 수 있는지에서 차이가 있습니다.

이번 프로젝트에서는 Windows 노트북을 사용하고, 코딩 에이전트와 협업하며, 기존 백엔드를 재사용해야 한다는 조건 때문에 Expo를 선택합니다.

. 중학생도 이해하는 네 가지 개발 방법

먼저 네 가지 방법을 음식점에 비유해 보겠습니다.

- PWA: 기존 음식점을 그대로 이용하되 휴대폰 홈 화면에 바로가기 아이콘을 만드는 것
- 웹 래핑: 기존 음식점을 포장 용기에 담아 새로운 상품처럼 판매하는 것
- 네이티브: iOS와 Android에 맞춰 음식점을 각각 새로 만드는 것
- React Native + Expo: 하나의 주방에서 만든 음식을 서로 다른 두 종류의 그릇에 담아 제공하는 것

① PWA(Progressive Web App)

Norway Travel App | Best App for Planning Trips

웹사이트를 앱처럼 사용하는 방법

기존 웹사이트에 필요한 설정 파일과 서비스 워커를 추가해 스마트폰 홈 화면에 설치하는 방식입니다.

가장 간단한 방식

PWA는 별도의 앱 빌드 과정이 거의 필요하지 않습니다.

예를 들어 AI 친구 웹사이트를 PWA로 만들면 사용자는 브라우저에서 '홈 화면에 추가'를 선택해 앱처럼 사용할 수 있습니다.

- 화면: 기존 웹 화면
- 설치: 브라우저에서 홈 화면에 추가
- 개발 도구: 웹 개발 환경
- iOS 빌드: 별도 앱 빌드 불필요
- 기기 기능: 브라우저가 허용하는 범위에서 사용

단점은 공식 앱스토어 배포 방식과 다르고, iOS와 Android에서 지원하는 기능에 차이가 있다는 것입니다.

② 웹 래핑(Web Wrapping)

Développer une application multiplateforme pour un fullstack

기존 웹사이트를 앱 안에 넣는 방법

앱이라는 껍데기 안에 브라우저(WebView)를 넣고 기존 웹사이트를 보여주는 방식입니다.

기존 웹 재활용

대표적인 기술이 Capacitor입니다.

예를 들어 기존 AI 친구 웹사이트를 그대로 활용하면서 앱 아이콘과 카메라 연결 기능 등을 추가할 수 있습니다.

- 화면: 기존 웹 화면
- 설치: APK, IPA 등 앱 파일 설치
- 개발 도구: Android Studio, Xcode 등
- iOS 빌드: Mac과 Xcode 필요
- 기기 기능: 플러그인을 통해 네이티브 기능 연결

장점은 이미 만들어 놓은 웹 화면을 상당 부분 재사용할 수 있다는 것입니다.

단점은 화면 자체가 여전히 웹 기술로 구성되어 있어 네이티브 앱과 사용 경험이 다를 수 있고, 앱 파일을 만들기 위한 플랫폼별 빌드 환경도 필요하다는 점입니다.

③ 네이티브(Native)

Native App Builder | No Code Mobile App Creator | Lastapp Ai

각 운영체제에 최적화해 만드는 방법

iOS는 Swift, Android는 Kotlin으로 각각 개발하는 방식입니다.

기기 기능 활용에 강점

네이티브 방식은 스마트폰 운영체제가 제공하는 기능을 직접 활용합니다.

- 화면: 운영체제의 네이티브 화면 요소
- 설치: 앱 파일 설치
- 개발 도구: Xcode, Android Studio
- iOS 빌드: Mac 필요
- 기기 기능: 폭넓은 활용 가능

장점은 성능과 기기 기능 활용 측면에서 유리하다는 것입니다.

단점은 iOS와 Android를 각각 개발할 경우 개발 및 유지보수 비용이 커진다는 점입니다.

참고로 Flutter처럼 하나의 코드 기반으로 두 플랫폼의 앱을 개발하는 기술도 있습니다. Flutter는 자체 렌더링 엔진을 사용하므로 Swift와 Kotlin으로 각각 만드는 방식과는 기술 구조가 다릅니다.

④ React Native + Expo

How to Start Expo For React Native | by Jimin Park | Medium

하나의 JavaScript 코드로 두 플랫폼의 앱을 만드는 방법

React Native로 화면을 만들고 Expo로 개발·실행·빌드·배포 과정을 편리하게 관리합니다.

이번 강의의 선택 기술

React Native는 JavaScript와 React를 이용해 모바일 앱을 개발하는 프레임워크입니다.

Expo는 React Native 개발에 필요한 도구를 모아 놓은 플랫폼입니다.

- 화면: 네이티브 화면 요소
- 설치: 앱 파일 설치
- 개발 도구: Node.js 중심의 개발 환경
- iOS 빌드: EAS 클라우드 빌드 활용 가능
- 기기 기능: 카메라, 사진첩, 위치, 알림 등 활용 가능

특히 Windows 환경에서도 iOS 앱 빌드를 클라우드에 맡길 수 있다는 것이 큰 장점입니다.

3. 네 가지 방법 한눈에 비교하기

Build Mobile Apps Without Coding Skills | Adalo

adalo.com

비교 항목	PWA	웹 래핑	네이티브	React Native + Expo
화면 구성	웹	웹뷰	네이티브	네이티브
홈 화면 설치	가능	가능	가능	가능
스토어 배포	일반적인 앱과 다름	가능	가능	가능
개발 코드	웹	웹 중심	플랫폼별	JavaScript 중심
Windows 개발	가능	Android 가능	Android 가능	가능
iOS 빌드	불필요	Mac 필요	Mac 필요	클라우드 가능
기기 기능	제한적	플러그인 활용	폭넓게 활용	모듈로 활용

비교 항목	PWA	웹 래핑	네이티브	React Native + Expo
개발 부담	낮음	낮음~중간	높음	중간

여기서 주의할 점이 있습니다.

React Native와 Expo가 모든 네이티브 기능을 자동으로 해결하는 것은 아닙니다. 추가적인 네이티브 모듈이나 설정이 필요할 수 있습니다. 또한 EAS 클라우드 빌드는 무료 플랜에서도 제한이 있으므로 사용량과 비용을 고려해야 합니다.

4. 왜 이번 프로젝트에서는 Expo를 선택했을까?

이번 강의에서 기술을 선택하는 기준은 세 가지입니다.

프로젝트의 세 가지 조건

조건 1. Windows 노트북

Mac을 별도로 구매하지 않고 개발해야 합니다.

조건 2. 코딩 에이전트 활용

AI 코딩 에이전트와 협업해 개발 생산성을 높입니다.

조건 3. 기존 백엔드 재사용

이미 구축한 서버와 데이터베이스는 그대로 사용합니다.

선택: React Native + Expo

개발 환경 부담을 줄이면서 iOS와 Android를 함께 지원할 수 있는 선택입니다.

다른 방법들이 잘못된 것은 아닙니다. 프로젝트의 목적과 조건에 따라 최적의 선택이 달라집니다.

예를 들어 단순히 웹사이트를 스마트폰에서 편리하게 이용하는 것이 목적이라면 PWA가 더 경제적일 수 있습니다.

반면 이번처럼 카메라, 사진첩, AI 대화, 향후 알림 기능까지 활용하려는 서비스라면 Expo가 더 적합합니다.

5. 나의 프로젝트에 적용한다면?

이번에 배운 기술 선택 기준을 현재 추진하는 프로젝트에 적용해 보겠습니다.

프로젝트	추천 방식	이유
AI 친구 에이전트	React Native + Expo	대화, 알림, 카메라 기능
MSM	React Native + Expo	사진, 음성, 일기 입력

프로젝트	추천 방식	이유
Travel Creator	React Native + Expo	여행 사진, 일정, 현장 기록
ASP 컨설팅	PWA 또는 Expo	간단한 진단은 PWA, 현장 자료 촬영은 Expo
AI 학습 도우미	PWA 또는 Expo	학습 자료 열람 중심이면 PWA도 적합

여기서 중요한 사업적 관점이 있습니다.

모든 서비스를 처음부터 모바일 앱으로 만들 필요는 없습니다.

서비스 초기에는 PWA나 웹으로 시장 반응을 확인하고, 사용자가 스마트폰 기능을 적극적으로 활용하기 시작할 때 Expo 기반 앱으로 확장하는 전략도 좋습니다.

개발 비용과 시장 검증 속도를 함께 고려해야 하기 때문입니다.

6. 객관식 퀴즈 정답 및 해설

정답: 2번

문제: PWA와 웹 래핑은 둘 다 화면을 웹으로 그립니다. 두 방법의 차이는 무엇인가요?

정답은 2번. 웹 래핑은 스토어에 올릴 앱 파일을 만들기 때문에, 내 컴퓨터에 Android Studio와 Xcode가 필요하다입니다.

이유를 정리하면 다음과 같습니다.

- PWA는 웹사이트에 설정을 추가해 홈 화면에 설치하는 방식입니다.
- 웹 래핑은 웹사이트를 앱 껍데기 안에 넣어 APK나 IPA 같은 앱 파일로 만듭니다.
- 따라서 웹 래핑은 플랫폼별 빌드 환경이 필요합니다.

나머지 선택지가 틀린 이유도 알아두시면 좋습니다.

- 1번: 두 방식의 설치 및 배포 설명이 반대로 되어 있습니다.
- 3번: 웹 래핑은 웹 화면을 네이티브 화면으로 바꾸지 않습니다.
- 4번: PWA가 스마트폰의 모든 기능을 사용할 수 있는 것은 아닙니다.

7. 반드시 기억해야 할 핵심 5가지

1. PWA는 기존 웹을 앱처럼 설치하는 가장 가벼운 방법입니다.
2. 웹 래핑은 기존 웹 화면을 앱 껍데기에 넣어 스토어용 앱 파일을 만드는 방법입니다.
3. 네이티브는 플랫폼의 기능을 폭넓게 활용할 수 있지만 개발 부담이 큽니다.
4. React Native는 JavaScript로 네이티브 화면을 만들고, Expo는 개발과 클라우드 빌드를 지원합니다.

5. 기술 선택은 유행이 아니라 사용자 요구, 개발 환경, 비용, 기기 기능 활용 정도에 따라 결정해야 합니다.

8. 오늘 강의 한 문장 정리

모바일 앱을 만드는 방법은 PWA, 웹 래핑, 네이티브, React Native + Expo 네 가지가 있으며, Windows 환경에서 기존 백엔드를 재사용하고 AI 에이전트와 협업해 iOS와 Android 앱을 개발하려면 Expo가 효과적인 선택입니다.

9. 다음 챕터를 위한 연결 개념

다음 강의부터는 Expo가 실제로 어떤 일을 하는지 배우게 됩니다.

특히 아래 세 가지를 구분해서 이해하시면 좋겠습니다.

- React Native: 모바일 앱의 화면과 기능을 만드는 기술
- Expo: React Native 앱의 개발과 실행을 편리하게 해주는 도구
- EAS: 앱을 클라우드에서 빌드하고 배포하는 서비스

이 세 가지의 역할을 이해하면 앞으로 모바일 앱 개발 구조가 훨씬 명확해집니다.

Chapter04

챕터 04. Expo의 정의와 작동 원리

핵심 주제: Expo의 구성 요소와 개발·빌드·배포 구조

학습 단계: 모바일 앱 개발 환경 이해

1. 강의의 목적

이번 강의의 핵심은 Expo가 무엇이고, React Native만 사용할 때보다 어떤 개발 작업을 줄여 주는지 이해하는 것입니다.

지난 챕터에서는 여러 모바일 앱 개발 방법 중 React Native와 Expo를 선택했습니다.

이번에는 Expo가 실제로 어떤 역할을 하는지 살펴봅니다.

Expo는 단순한 앱 제작 도구가 아닙니다. 모바일 앱을 개발하고, 스마트폰에서 실행하고, 앱 파일로 빌드하고, 스토어에 배포하는 과정을 지원하는 통합 개발 환경입니다.

특히 다음 다섯 가지를 이해해야 합니다.

1. Expo가 제공하는 주요 구성 요소

2. Expo를 사용하면 개발 작업이 어떻게 줄어드는가?
3. Expo Go와 개발용 빌드의 차이
4. EAS 클라우드 빌드가 필요한 이유
5. Expo SDK 버전 관리의 중요성

. 중학생도 이해하는 Expo의 개념

① Expo란 무엇인가?

먼저 React Native와 Expo의 관계부터 이해하겠습니다.

- React Native: 스마트폰 앱의 화면과 기능을 만드는 기술
- Expo: React Native 앱을 쉽게 개발하고 테스트하고 빌드하도록 도와주는 도구 모음

비유하면 다음과 같습니다.

자동차를 만든다고 가정해 보겠습니다.

- React Native는 자동차의 엔진과 차체를 만드는 기술입니다.
- Expo는 자동차를 만들 때 필요한 공구, 설계도, 조립 시설, 시험 주행장을 한곳에 제공하는 종합 자동차 제작 시스템입니다.

React Native만으로도 앱을 만들 수 있지만 화면 이동, 카메라, 권한 설정, 빌드 환경 등을 직접 구성해야 합니다.

Expo는 이런 작업을 미리 준비된 도구로 해결하도록 도와줍니다.

React Native와 Expo의 관계

Expo

개발 · 기기 기능 · 설정 · 빌드 · 배포 지원

React Native

JavaScript 코드로 모바일 네이티브 화면 구현

iOS

Android

② Expo가 제공하는 주요 구성 요소 6가지

Expo는 여러 도구를 하나로 묶어 제공합니다.

구성 요소	하는 일	쉽게 이해하면
Expo CLI	개발 명령 실행	작업 지시 도구
Expo SDK	카메라, 사진첩 등 기능 제공	기능별 공구 상자
Expo Router	화면 이동 관리	앱의 길 안내
App Config	앱 설정 관리	앱의 기본 설계서
Expo Go	스마트폰에서 테스트	시험 운전 차량
Expo Services(EAS)	클라우드 빌드와 배포	외부 제작 공장

각각을 조금 더 자세히 살펴보겠습니다.

1. Expo CLI와 개발 서버

CLI는 Command Line Interface의 약자입니다.

쉽게 말하면 명령어를 입력해 개발 도구를 실행하는 방식입니다.

대표적인 명령어는 다음과 같습니다.

```
npx expo start
```

이 명령을 실행하면 개발 서버가 시작됩니다.

개발 서버 안에는 Metro라는 번들러가 작동합니다.

Metro는 작성한 TypeScript 또는 JavaScript 코드를 스마트폰에서 실행할 수 있도록 필요한 파일로 묶어 전달합니다.

파일을 저장하면 바뀐 부분을 다시 보내므로 스마트폰 화면이 빠르게 바뀝니다.

- . Expo SDK

SDK는 스마트폰의 다양한 기능을 쉽게 사용할 수 있도록 만든 라이브러리 모음입니다.

예를 들어 다음과 같습니다.

라이브러리	기능
expo-camera	앱 내부 카메라 실행
expo-image-picker	사진첩에서 이미지 선택
expo-notifications	알림 기능
expo-file-system	파일 읽기와 저장

원래라면 iOS와 Android의 기능을 각각 연결해야 할 수 있지만 Expo SDK를 이용하면 하나의 코드로 두 플랫폼을 지원하기 쉬워집니다.

3. Expo Router

앱의 화면 이동을 관리하는 도구입니다.

예를 들어 AI 친구 앱에 다음 화면이 있다고 가정하겠습니다.

- 로그인 화면
- 메인 채팅 화면
- 설정 화면
- 대화 기록 화면

Expo Router는 화면마다 파일을 만들고, 파일의 위치와 이름을 기반으로 화면 이동을 관리할 수 있게 해줍니다.

4. App Config

앱의 기본 설정을 관리합니다.

대표적인 파일이 `app.json`입니다.

여기에 다음 정보를 설정합니다.

- 앱 이름
- 앱 아이콘
- 패키지 이름
- 카메라 권한 설명
- 알림 권한 설정

즉, 앱의 기본 설계도를 한곳에서 관리하는 것입니다.

5. Expo Go

개발 중인 앱을 실제 스마트폰에서 실행해 보는 테스트용 앱입니다.

노트북에서 개발 서버를 실행하고 스마트폰의 Expo Go로 QR 코드를 찍으면 개발 중인 앱이 실행됩니다.

6. Expo Services(EAS)

EAS는 Expo Application Services의 약자입니다.

클라우드에서 앱을 빌드하고 스토어에 제출하거나 JavaScript 코드를 업데이트하는 기능을 제공합니다.

- EAS Build: 앱 파일 빌드
- EAS Submit: 앱스토어 제출 지원

- EAS Update: JavaScript 코드와 이미지 등의 업데이트

강의 자료에서 Expo Services라는 표현을 사용하더라도, 실제 프로젝트에서 만나는 `eas build`, `eas.json` 등의 명칭은 그대로 기억하시면 됩니다.

3. Expo를 사용하면 무엇이 편해지는가?

가장 큰 차이는 개발자가 직접 설치하고 관리해야 하는 도구가 줄어든다는 점입니다.

① 일반 React Native 방식

일반적인 React Native 프로젝트에서는 다음 작업을 직접 관리해야 합니다.

- Android Studio 설치
- Android SDK 설정
- Mac과 Xcode 준비
- iOS와 Android 네이티브 프로젝트 관리
- 카메라 권한을 플랫폼별로 각각 설정
- 네이티브 라이브러리 버전 관리
- 플랫폼별 빌드와 서명

② Expo 방식

Expo를 사용하면 많은 작업을 간소화할 수 있습니다.

- 노트북에 Node.js 설치
- Expo 프로젝트 생성
- Expo Go 설치
- QR 코드로 스마트폰 연결
- 앱 화면 개발
- 필요할 때 EAS 클라우드 빌드

개발 방식 비교

일반 React Native

개발자가 직접 준비하고 관리할 항목이 많습니다.

Android SDK

Xcode / Mac

네이티브 설정

권한 관리

버전 관리

빌드 / 서명

Expo

개발 환경과 설정을 통합해 관리합니다.

Node.js

Expo CLI

Expo SDK

app.json

Expo Go

EAS Build

③ CNG(Continuous Native Generation)

이 부분은 조금 생소하지만 중요합니다.

일반적인 React Native 프로젝트는 `android/`와 `ios/`라는 폴더를 직접 관리합니다.

그런데 Expo는 `app.json`에 설정을 적어 놓으면 빌드 과정에서 필요한 네이티브 프로젝트 폴더를 생성할 수 있습니다.

이 방식을 CNG라고 합니다.

쉽게 비유하면 다음과 같습니다.

- 일반 방식: 집을 지을 때마다 벽돌과 배관을 직접 설계하고 관리합니다.
- Expo 방식: 설계도에 필요한 내용을 적으면 건축 시스템이 벽과 배관을 만들어 줍니다.

예를 들어 카메라 권한을 추가한다고 가정해 보겠습니다.

일반 방식에서는 Android와 iOS 설정 파일을 각각 수정해야 할 수 있습니다.

Expo에서는 `app.json`에 권한 관련 설정을 작성하고 빌드 과정에서 반영하도록 할 수 있습니다.

필요할 때 다음 명령으로 네이티브 폴더를 생성합니다.

```
npx expo prebuild
```

따라서 일반적인 Expo 프로젝트에서는 `android/`, `ios/` 폴더를 직접 수정하기보다는 설정 파일과 플러그인을 중심으로 관리합니다.

이렇게 하면 코딩 에이전트가 변경해야 할 파일도 줄어들어 코드 검토가 쉬워집니다.

4. 컴파일이 필요한 부분과 필요 없는 부분

이번 강의에서 기술적으로 가장 중요한 개념입니다.

모바일 앱은 크게 두 부분으로 나뉩니다.

1. 네이티브 부분
2. JavaScript 또는 TypeScript 코드 부분

네이티브 부분은 플랫폼에 맞게 컴파일해야 합니다.

반면 JavaScript와 TypeScript 코드는 번들로 묶어 실행할 수 있습니다.

모바일 앱을 구성하는 두 부분

네이티브 부분

카메라 연결, 운영체제 기능, 네이티브 라이브러리

플랫폼별 컴파일 필요

JavaScript / TypeScript 부분

화면 구성, 버튼 동작, 로그인, API 호출

번들링으로 전달 가능

그렇다면 Expo는 이 두 부분을 어떻게 처리할까요?

두 가지 방법을 사용합니다.

① 개발할 때: Expo Go

Expo Go는 자주 사용하는 네이티브 기능을 미리 컴파일해 넣어 둔 앱입니다.

내가 작성한 JavaScript 코드는 노트북의 개발 서버에서 번들로 만들어집니다.

스마트폰은 이 번들을 받아 Expo Go 안에서 실행합니다.

따라서 매번 앱 전체를 새로 빌드하지 않아도 됩니다.

② 다른 사람에게 배포할 때: EAS Build

다른 사람의 스마트폰에 설치할 수 있는 앱 파일을 만들려면 네이티브 부분을 포함해 빌드해야 합니다.

이때 EAS Build를 사용합니다.

1. 내 프로젝트를 Expo 클라우드로 보냅니다.
2. 클라우드의 빌드 환경이 Android 또는 iOS용 네이티브 코드를 컴파일합니다.
3. 앱에 서명합니다.
4. APK, AAB 또는 IPA 등의 결과물을 만듭니다.
5. 설치 링크나 스토어 제출용 파일을 받을 수 있습니다.

Expo의 개발부터 배포까지

내 Windows 노트북

코드 작성 · npx expo start

개발 단계

Expo Go

번들을 받아 즉시 실행

빌드 단계

EAS Build

네이티브 컴파일 및 서명

앱 파일 생성 및 배포

Android · iOS → 앱스토어 또는 직접 설치

이것이 Windows 노트북만으로 iOS 앱 개발을 진행할 수 있는 핵심 원리입니다.

5. Expo Go와 개발용 빌드의 차이

여기서는 두 가지를 반드시 구분해야 합니다.

Expo Go는 일반적인 개발 테스트용 앱이고, 개발용 빌드는 내 프로젝트에 필요한 네이티브 기능을 포함한 전용 앱입니다.

기능	Expo Go	개발용 빌드
화면 구성	가능	가능
백엔드 API 호출	가능	가능
로그인	가능	가능
AI 스트리밍	가능	가능
카메라 촬영	가능	가능
사진첩 선택	가능	가능
로컬 알림	가능	가능

기능	Expo Go	개발용 빌드
서버 푸시 알림	제한적	가능
카카오 로그인 SDK	불가	가능
별도 네이티브 라이브러리	제한적	가능

참고로 푸시 알림은 Android의 Expo Go에서 SDK 53부터 지원되지 않는다는 점을 기억하시면 좋겠습니다. 실제 푸시 알림을 구현할 때는 개발용 빌드와 플랫폼별 설정을 준비해야 합니다.

이번 강의의 AI 친구 대화 앱은 다음 기능이 중심입니다.

- 화면 표시
- 백엔드 호출
- 로그인
- AI 답변 스트리밍
- 카메라 촬영
- 사진첩에서 이미지 선택

이 기능들은 Expo Go에서 확인할 수 있으므로 초기 실습에는 Expo Go로 충분합니다.

6. SDK 버전 관리가 중요한 이유

Expo는 SDK라는 버전 체계를 사용합니다.

예를 들어 프로젝트가 Expo SDK 58을 사용한다면, 프로젝트에서 사용하는 React Native와 Expo 라이브러리도 SDK 58에 맞는 조합으로 관리해야 합니다.

쉽게 비유하면 자동차 부품을 조립할 때 같은 규격의 부품을 사용해야 하는 것과 같습니다.

서로 맞지 않는 부품을 사용하면 조립이 제대로 되지 않습니다.

Expo도 마찬가지입니다.

- 프로젝트의 Expo SDK 버전
- React Native 버전
- Expo SDK 라이브러리 버전
- 스마트폰에 설치된 Expo Go의 지원 버전

이 네 가지가 서로 맞아야 합니다.

강의 자료에서는 SDK 58을 기준으로 실습한다고 설명하고 있습니다.

실제 개발에서는 프로젝트의 `package.json`에 기록된 Expo 버전을 확인하고, 설치된 Expo Go가 해당 SDK를 지원하는지 확인해야 합니다.

라이브러리를 설치할 때도 일반적인 `npm install`보다 다음 명령을 우선 사용합니다.

```
npx expo install expo-camera
```

이 명령은 현재 프로젝트의 Expo SDK와 호환되는 라이브러리 버전을 선택하는 데 도움을 줍니다.

7. 나의 프로젝트에 적용하기

이번 강의는 현재 추진하는 AI 서비스들의 모바일 확장 기반이 됩니다.

프로젝트	Expo 활용 방향
AI 친구 에이전트	대화, 사진 입력, 일정 등록
MSM	일기 작성, 사진 및 음성 입력
Travel Creator	여행 사진 촬영, 일정 기록
ASP 컨설팅	현장 자료 촬영, AI 분석 요청

특히 MSM 프로젝트를 생각해 보겠습니다.

사용자가 스마트폰으로 일기를 쓰거나 사진을 찍으면 Expo SDK로 자료를 가져옵니다.

그 자료를 기존 백엔드에 보내고, AI가 분석한 뒤 웹소설이나 자서전 형태로 변환할 수 있습니다.

즉, Expo는 단순히 화면을 모바일로 옮기는 기술이 아니라 AI 서비스가 실제 생활의 정보와 연결되도록 만드는 기술입니다.

8. 오늘 강의에서 반드시 기억할 핵심 7가지

1. Expo는 React Native 앱 개발에 필요한 도구와 라이브러리, 클라우드 서비스를 통합한 프레임워크입니다.
2. Expo CLI는 개발 서버를 실행하고, Metro는 JavaScript 코드를 번들로 묶어 스마트폰에 전달합니다.
3. Expo SDK는 카메라, 사진첩, 알림 등 스마트폰 기능을 활용하도록 도와줍니다.
4. Expo Router는 앱 화면 간 이동을 관리하고, `app.json`은 앱의 기본 설정을 관리합니다.

5. Expo Go는 미리 컴파일된 네이티브 기능으로 개발 중인 앱을 실행하는 테스트 환경입니다.
6. EAS Build는 클라우드에서 네이티브 코드를 컴파일하고 서명해 앱 파일을 만듭니다.
7. Expo Go에 없는 네이티브 기능을 사용하려면 개발용 빌드가 필요합니다.

9. 객관식 퀴즈 정답 및 해설

정답: 3번

문제: Expo Go로는 확인할 수 없고 개발용 빌드가 필요한 기능은 무엇인가요?

정답은 3. 카카오 로그인 SDK로 로그인하기입니다.

이유는 카카오 로그인 SDK가 Expo Go에 기본 포함된 네이티브 기능이 아니기 때문입니다.

- 1번 카메라 촬영: Expo Go에서 지원합니다.
- 번 로컬 알림 예약: Expo Go에서 지원합니다.
- 3번 카카오 로그인 SDK: 별도의 네이티브 SDK를 포함한 개발용 빌드가 필요합니다.
- 4번 AI 답변 스트리밍: 일반적인 서버 통신과 화면 업데이트이므로 Expo Go에서 가능합니다.

핵심은 기능 자체가 복잡한지 여부가 아니라 필요한 네이티브 라이브러리가 Expo Go에 포함되어 있는지 여부입니다.

10. 오늘 강의 한 문장 정리

Expo는 React Native 앱 개발에 필요한 도구와 스마트폰 기능 라이브러리를 통합하고, Expo Go로 빠르게 테스트하며 EAS 클라우드 빌드로 앱 파일을 생성할 수 있게 해주는 모바일 앱 개발 플랫폼입니다.

11. 다음 챕터를 이해하기 위한 핵심 연결

이번 챕터에서 배운 내용을 하나의 흐름으로 기억하시면 좋겠습니다.

Expo 모바일 앱 개발 흐름

1. Node.js 설치

   Windows에서 Expo 개발 환경 준비

2. Expo 프로젝트 생성

   앱의 기본 구조와 설정 준비

3. npx expo start

   개발 서버와 Metro 실행

4. Expo Go로 스마트폰 연결

   QR 코드로 개발 중인 앱 실행

5. 화면 및 기능 개발

   AI 코딩 에이전트와 함께 구현하고 테스트

6. EAS Build

   클라우드에서 설치 가능한 앱 파일 생성

7. 스토어 배포

   사용자가 다운로드하고 설치하도록 출시

이번 챕터에서 가장 중요한 것은 Expo Go와 EAS Build의 역할을 구분하는 것입니다.

Expo Go는 개발 중인 앱을 실행하는 도구이고, EAS Build는 다른 사람에게 설치시킬 수 있는 앱 파일을 만드는 도구입니다.

Chapter05

챕터 05. 모바일 앱 개발을 위한 시작 상태 맞추기

핵심 주제: 개발 환경 구축과 프로젝트 초기 설정

학습 단계: 실습 준비 및 개발 환경 구성

1. 강의의 목적

이번 강의의 핵심은 Expo로 모바일 앱을 개발하기 전에 필요한 개발 환경을 준비하고, 다른 사람과 동일한 조건에서 실습할 수 있도록 프로젝트의 시작 상태를 맞추는 것입니다.

앞선 강의에서 Expo의 개념과 작동 원리를 배웠다면, 이번에는 실제 개발을 시작하기 위한 준비 과정입니다.

오늘 준비할 것은 크게 5가지입니다.

1. Node.js 설치 및 버전 확인
2. 스마트폰에 Expo Go 설치
3. 기존 백엔드 연결 상태 확인

4. GitHub 템플릿으로 모바일 앱 저장소 만들기
5. 코딩 에이전트에 Expo 스킬 설치하기

여기서 중요한 원칙이 있습니다.

이번 프로젝트는 기존 백엔드는 그대로 사용하고, 모바일 앱의 화면만 새롭게 개발합니다.

따라서 백엔드를 새로 만들 필요는 없습니다.

. 중학생도 이해하는 개발 환경 준비

① Node.js 설치하기

swz22 (John Kim) · GitHub

Node.js

JavaScript 코드를 컴퓨터에서 실행할 수 있게 해주는 프로그램입니다.

Expo는 JavaScript와 TypeScript를 기반으로 작동합니다.

그런데 JavaScript는 원래 웹 브라우저에서 실행되는 언어입니다. Node.js를 설치하면 브라우저 밖에서도 JavaScript를 실행할 수 있습니다.

쉽게 비유하면 다음과 같습니다.

- JavaScript: 자동차의 연료
- Node.js: 연료를 실제로 사용할 수 있게 해주는 엔진
- Expo: 엔진을 활용해 자동차를 만드는 도구

이번 강의에서는 Node.js 22.13 이상이 필요하며 LTS 24가 권장됩니다.

Windows에서는 다음 명령으로 설치할 수 있습니다.

```
winget install OpenJS.NodeJS.LTS
```

설치 후에는 터미널을 새로 열고 다음 명령으로 확인합니다.

```
node -v
npm -v
```

여기서 알아야 할 용어가 있습니다.

용어	의미
Node.js	JavaScript 실행 환경
npm	JavaScript 패키지를 설치하고 관리하는 도구
npx	패키지에 포함된 명령을 실행하는 도구
LTS	장기간 안정적인 지원을 제공하는 버전

특히 `npm`과 `npx`는 앞으로 Expo 개발에서 자주 사용하게 됩니다.

② 스마트폰에 Expo Go 설치하기

Expo Go - Developer Tools App | MWM

Expo Go

노트북에서 개발 중인 모바일 앱을 실제 스마트폰에서 실행하는 테스트용 앱입니다.

설치 방법은 간단합니다.

- Android: Google Play 스토어
- iPhone: Apple App Store

설치 후 Expo Go의 SDK 버전이 프로젝트에서 사용하는 SDK 58과 일치하는지 확인해야 합니다.

왜 중요할까요?

프로젝트가 SDK 58을 사용하는데 Expo Go가 해당 버전을 지원하지 않으면 앱을 실행할 수 없기 때문입니다.

③ 기존 백엔드 연결 상태 확인하기

이번 프로젝트에서 중요한 구조입니다.

앱은 새롭게 만들지만 백엔드는 그대로 사용합니다.

예를 들어 AI 친구 서비스가 이미 웹에서 작동하고 있다면 다음과 같은 기능은 기존 백엔드가 담당합니다.

- 사용자 로그인
- AI와 대화
- 대화 기록 저장
- 사용자 기억 관리
- 크레딧 관리

모바일 앱은 이 백엔드에 API 요청을 보내고 결과를 받아 화면에 보여줍니다.

기존 웹 서비스와 신규 모바일 앱의 관계

기존 웹

웹 화면

신규 앱

Expo 화면

동일한 백엔드 API 사용

기존 백엔드 서버

로그인 · AI 대화 · 기억 · 크레딧

기존 데이터베이스

백엔드가 정상적으로 작동하는지 확인하는 방법은 다음과 같습니다.

브라우저 주소창에 아래 주소를 입력합니다.

```
https://내-백엔드-주소/health
```

정상이라면 다음과 같은 결과가 나옵니다.

```
{
  "status": "ok"
}
```

여기서 중요한 것은 백엔드가 공개 주소로 접근 가능해야 한다는 것입니다.

내 컴퓨터에서만 접근할 수 있는 `localhost` 주소는 외부 스마트폰에서 직접 접근할 수 없습니다.

백엔드가 Supabase 로그인을 사용한다면 익명 로그인 설정도 확인해야 합니다.

- Supabase 프로젝트 접속
- Authentication
- Sign In / Providers
- Allow anonymous sign-ins 활성화

익명 로그인은 회원가입 없이 임시 사용자로 서비스를 이용하도록 하는 기능입니다.

3. GitHub 템플릿으로 내 앱 저장소 만들기

① 왜 별도 저장소를 만드는가?

이번 프로젝트에서는 백엔드와 모바일 앱을 별도의 저장소로 관리합니다.

예를 들어 다음과 같습니다.

구분	기존 백엔드	신규 모바일 앱
저장소	friend-agent	friend-agent-mobile
주요 언어	Python	JavaScript / TypeScript
개발 환경	Python 중심	Node.js + Expo
담당 기능	AI 처리, 데이터, API	스마트폰 화면, 사용자 입력

쉽게 비유하면 백엔드는 식당 주방이고 모바일 앱은 고객이 사용하는 주문 단말기입니다.

주방과 주문 단말기는 서로 연결되지만 별도로 관리하는 것이 효율적입니다.

② 템플릿이란?

Creating a theme - Inkdrop API Reference

GitHub 템플릿

이미 만들어진 프로젝트의 기본 구조와 파일을 복사해 새로운 프로젝트를 시작할 수 있도록 하는 기능입니다.

이번 강의에서는 다음 템플릿을 사용합니다.

`88chacha/friend-agent-mobile-template`

이 템플릿을 복사해 내 계정에 `friend-agent-mobile`이라는 저장소를 만듭니다.

직접 실행할 명령은 다음과 같습니다.

```
gh repo create friend-agent-mobile \
  --template 88chacha/friend-agent-mobile-template \
  --public --clone

cd friend-agent-mobile

npm install
```

각 명령의 의미를 이해해 보겠습니다.

명령어	역할
gh repo create	GitHub 저장소 생성
--template	복사할 원본 템플릿 지정

명령어	역할
--public	공개 저장소로 생성
--clone	생성한 저장소를 내 컴퓨터로 내려받기
cd	해당 프로젝트 폴더로 이동
npm install	필요한 패키지 설치

여기서 `npm install`은 매우 중요합니다.

템플릿에는 앱을 실행하는 데 필요한 여러 라이브러리의 목록이 들어 있습니다.

`package.json`에 기록된 패키지를 내 컴퓨터에 설치하는 과정입니다.

쉽게 말하면 새 컴퓨터에 프로그램을 설치할 때 필요한 부품을 한꺼번에 준비하는 것입니다.

③ README.md와 AGENTS.md 확인하기

프로젝트를 내려받으면 두 파일을 확인해야 합니다.

README.md

프로젝트가 무엇인지, 어떻게 실행하는지 알려주는 안내서입니다.

AGENTS.md

코딩 에이전트가 해당 프로젝트에서 지켜야 할 작업 규칙입니다.

예를 들어 다음과 같은 내용이 포함될 수 있습니다.

- 프로젝트의 폴더 구조
- 패키지 설치 방법
- 비밀정보 관리 원칙
- 빌드 및 테스트 방법
- 코딩 에이전트의 작업 범위

이 파일은 단순한 설명서가 아닙니다.

앞으로 Cursor나 Claude Code 같은 코딩 에이전트에게 작업을 맡길 때 프로젝트의 일관성을 유지하도록 도와주는 기준입니다.

4. 코딩 에이전트에 Expo 스킬 설치하기

① 스킬(Skill)이란?

스킬은 AI 코딩 에이전트가 특정 작업을 잘 수행하도록 제공하는 작업 지침서입니다.

예를 들어 AI에게 모바일 앱을 만들라고 요청하면 AI는 학습한 지식에 따라 코드를 작성합니다.

그런데 Expo는 계속 업데이트되므로 AI가 과거 버전의 방법을 사용할 수도 있습니다.

이때 Expo 공식 스킬을 제공하면 AI가 다음과 같은 지침을 참고할 수 있습니다.

- Expo Router를 사용하는 방법
- Expo SDK에 맞는 라이브러리 설치 방법
- SDK 업그레이드 방법
- 앱 빌드와 배포 방법

즉, 스킬은 AI에게 특정 분야의 작업 매뉴얼을 제공하는 것입니다.

② 코딩 에이전트별 설치 방법

코딩 에이전트	설치 방법
Claude Code	공식 Expo 플러그인 설치
Codex	공식 마켓플레이스 플러그인 설치
Cursor 등	skills CLI 활용

대표적인 설치 명령은 다음과 같습니다.

```
# Claude Code
claude plugin install expo@claude-plugins-official --scope project

# Codex
codex plugin add expo@openai-curated

# 기타 코딩 에이전트
npx skills@latest add expo/skills --skill '*'
```

여기서 `--scope project`는 프로젝트 단위로 설치한다는 의미입니다.

즉, 내 컴퓨터 전체가 아니라 현재 앱 프로젝트에 설치하는 것입니다.

이렇게 하면 프로젝트의 설정을 다른 사람과 공유하기도 쉬워집니다.

설치가 끝나면 에이전트를 다시 시작하고 다음과 같이 요청해 확인합니다.

```
설치된 Expo 스킬 목록을 보여 줘.
```

③ 왜 공식 스킬을 사용해야 할까?

AI는 코드를 작성하는 능력뿐 아니라 올바른 개발 방법을 선택하는 능력도 중요합니다.

특히 모바일 앱은 운영체제별 차이와 SDK 버전 문제가 있습니다.

따라서 AI가 임의로 코드를 작성하는 것보다 공식 문서와 스킬을 참고하도록 하는 것이 안전합니다.

이번 강의의 중요한 교훈은 다음과 같습니다.

AI 코딩 에이전트의 성능은 모델 자체의 능력뿐 아니라, 어떤 프로젝트 규칙과 전문 지식을 제공하느냐에 따라서도 달라집니다.

5. 전체 실습 준비 과정 정리

이번 챕터에서 진행한 준비 과정을 하나의 흐름으로 정리하겠습니다.

모바일 앱 개발 준비 5단계

1. 개발 환경 준비

   Node.js 22.13 이상 설치 및 npm 확인

2. 스마트폰 준비

   Expo Go 설치 및 SDK 58 지원 여부 확인

3. 백엔드 준비

   공개 API 주소와 health 상태 확인, 필요한 경우 Supabase 익명 로그인 활성화

4. 앱 프로젝트 준비

   GitHub 템플릿 복사, 저장소 생성, npm install

5. AI 개발 환경 준비

   Expo 공식 스킬 설치 및 에이전트에서 확인

6. 나의 프로젝트에 적용하기

이번 강의에서 배운 개발 환경 구축 방식은 앞으로 추진할 여러 프로젝트에도 재사용할 수 있습니다.

프로젝트	재사용할 준비 요소
AI 친구 에이전트	Expo 템플릿, 기존 백엔드 연결
MSM	별도 모바일 저장소, 사진·음성 기능 준비
Travel Creator	카메라, 사진첩, 일정 관리 라이브러리

프로젝트	재사용할 준비 요소
ASP	기존 컨설팅 백엔드와 모바일 화면 분리

특히 중요한 것은 프로젝트마다 백엔드와 프런트엔드의 역할을 구분하는 것입니다.

앞으로 서비스를 여러 개 만들 때도 이 원칙을 지키면 프로젝트 간 기술과 운영 체계를 재사용하기 쉬워집니다.

또한 GitHub 템플릿과 AGENTS.md를 잘 관리하면 새로운 프로젝트를 시작할 때마다 개발 환경을 처음부터 구축할 필요가 줄어듭니다.

7. 반드시 기억해야 할 핵심 7가지

1. Node.js는 JavaScript를 컴퓨터에서 실행하기 위한 환경입니다.
2. npm은 패키지를 설치하고 관리하며, npx는 패키지 명령을 실행합니다.
3. Expo Go는 개발 중인 앱을 실제 스마트폰에서 확인하는 테스트 도구입니다.
4. 기존 백엔드와 신규 모바일 앱은 별도 저장소로 관리할 수 있습니다.
5. `npm install`은 프로젝트 실행에 필요한 패키지를 설치하는 명령입니다.
6. README.md는 프로젝트 설명서이고 AGENTS.md는 코딩 에이전트의 작업 규칙입니다.
7. Expo 스킬은 AI 코딩 에이전트가 공식적인 Expo 개발 방법을 따르도록 돕습니다.

8. 오늘 강의 한 문장 정리

모바일 앱 개발을 시작하려면 Node.js와 Expo Go를 준비하고, 기존 백엔드와 분리된 GitHub 앱 저장소를 만든 다음, Expo 공식 스킬을 코딩 에이전트에 설치해 일관된 개발 환경을 구축해야 합니다.

9. AI 개발자 관점에서 얻어야 할 교훈

이번 강의에서 특히 주목할 부분은 코드를 작성하기 전에 개발 환경과 작업 규칙을 먼저 준비한다는 것입니다.

비개발자가 AI 코딩 에이전트와 함께 서비스를 개발할 때도 마찬가지입니다.

AI에게 단순히 "앱을 만들어 줘"라고 요청하는 것보다 다음 세 가지를 먼저 준비하는 편이 좋습니다.

- 환경: 어떤 언어와 도구, 버전을 사용할 것인가?
- 구조: 백엔드와 프런트엔드를 어떻게 분리할 것인가?

- 규칙: AI가 어떤 문서와 원칙을 따르면서 작업할 것인가?

이 세 가지가 정리되어 있으면 AI가 만든 결과물을 검토하고 수정하는 일도 훨씬 쉬워집니다.

오늘의 핵심 교훈: 좋은 AI 개발은 좋은 코드를 요청하는 것에서 시작하는 것이 아니라, AI가 올바른 코드를 만들 수 있는 환경과 규칙을 설계하는 것에서 시작합니다.

Chapter06

챕터 06. 모바일 앱 템플릿의 구조와 작동 원리

핵심 주제: 앱과 백엔드의 역할 분리

학습 단계: 앱 구조와 데이터 흐름 이해

1. 강의의 목적

이번 강의의 핵심은 모바일 앱은 화면을 담당하고, 데이터 처리와 판단은 기존 백엔드가 담당하도록 구조를 설계하는 것입니다.

앞선 강의에서는 모바일 앱을 개발하기 위한 환경을 준비했습니다. 이번에는 템플릿의 내부 구조를 살펴보고, 앱을 켜서 AI에게 메시지를 보내고 답변을 받기까지 어떤 일이 일어나는지 이해합니다.

오늘 반드시 이해해야 할 내용은 네 가지입니다.

1. 모바일 앱의 파일과 폴더가 각각 어떤 역할을 하는가?
2. 앱과 백엔드, Supabase는 어떤 순서로 통신하는가?
3. AI의 답변은 어떻게 실시간으로 조금씩 화면에 표시되는가?
4. 앱에 공개해도 되는 정보와 절대 넣으면 안 되는 비밀정보는 무엇인가?

. 앱은 화면, 백엔드는 데이터와 판단을 담당합니다.

이번 강의에서 가장 중요한 설계 원칙입니다.

기존 웹 서비스가 작동하고 있다면 모바일 앱을 만든다고 해서 백엔드까지 새로 만들 필요는 없습니다.

기존 웹 화면 대신 스마트폰 화면을 만들고, 두 화면이 동일한 백엔드 API를 사용하면 됩니다.

웹과 모바일 앱의 공통 구조

웹 화면

기존 사용자 인터페이스

모바일 앱

새롭게 만든 사용자 인터페이스

동일한 API를 통해 통신

기존 백엔드

로그인 · AI 판단 · 대화 처리 · 기억 관리

데이터베이스 및 Supabase

사용자 정보 · 대화 기록 · 서비스 데이터

쉽게 비유하면 다음과 같습니다.

- 웹 화면: 식당의 기존 주문 단말기
- 모바일 앱: 새로 만든 스마트폰 주문 앱
- 백엔드: 식당 주방
- 데이터베이스: 주문 내역과 고객 정보를 보관하는 장부

주문 단말기가 바뀌어도 주방의 조리 방식과 고객 정보 관리 시스템은 그대로 사용할 수 있습니다.

이것이 이번 모바일 앱 개발의 핵심입니다.

3. 모바일 앱 템플릿의 파일 구조

이번 템플릿은 기능별로 파일을 나누어 관리합니다.

처음에는 파일 이름이 복잡해 보이지만, 각 파일의 역할을 이해하면 전체 구조가 쉽게 보입니다.

① 전체 파일 구조

```
friend-agent-mobile/
│
└── src/
    │
    ├── app/
    │   ├── _layout.tsx
    │   ├── index.tsx
    │   └── character.tsx
```

```
    │
    ├── lib/
    │   ├── api.ts
    │   ├── session.tsx
    │   └── supabase.ts
    │
    └── components/
        ├── 버튼 관련 파일
        ├── 로딩 화면
        └── 오류 화면
```

세 개의 폴더를 이해하면 됩니다.

폴더	담당 역할	쉽게 비유하면
app	화면 구성과 이동	매장과 각 공간
lib	API, 로그인, 설정 등 공통 기능	매장 운영 시스템
components	여러 화면에서 공통으로 쓰는 요소	공통 집기와 도구

② src/app: 화면을 만드는 곳

Expo Router vs React Navigation: Which One Should You Use in 2026? - DEV Community

Expo Router

앱 화면을 파일 이름과 폴더 구조에 연결해 주는 도구입니다.

`src/app/` 폴더 안의 파일 하나가 화면 하나가 됩니다.

파일	역할
`_layout.tsx`	앱 전체 틀과 화면 공통 설정
`index.tsx`	메인 대화 화면
`character.tsx`	캐릭터 선택 화면

특히 파일 이름이 화면 주소가 된다는 점이 중요합니다.

예를 들어 다음 파일을 만들면,

```
src/app/memories.tsx
```

자동으로 다음 화면 주소가 만들어집니다.

```
/memories
```

즉, 별도로 복잡한 주소 설정을 하지 않아도 파일 구조를 통해 화면을 관리할 수 있습니다.

_layout.tsx가 하는 일

앱 전체의 공통 틀을 담당합니다.

- 로그인 상태 관리
- 화면 간 공통 정보 전달
- 화면을 쌓아 보여 주는 내비게이션 관리

index.tsx가 하는 일

사용자가 앱을 열었을 때 기본적으로 보이는 대화 화면입니다.

캐릭터가 선택되지 않았다면 캐릭터 선택 화면으로 이동시킵니다.

character.tsx가 하는 일

사용자가 AI 친구의 캐릭터를 선택하는 화면입니다.

③ src/lib: 앱의 공통 기능을 담당하는 곳

이 폴더는 화면 자체보다는 앱이 정상적으로 작동하도록 지원하는 기능을 모아 둔 곳입니다.

파일	역할
api.ts	백엔드 주소, API 요청, 답변 스트리밍
session.tsx	앱 실행 시 세션 및 설정 관리
supabase.ts	Supabase 연결 및 로그인 처리

특히 `api.ts`가 중요합니다.

화면 파일에서 백엔드에 직접 요청하지 않고 `api.ts`를 거치도록 설계했습니다.

왜 그럴까요?

모든 화면에서 백엔드 주소를 직접 입력하고 로그인 토큰을 붙인다면, 주소가 바뀌거나 인증 방식이 변경될 때 여러 파일을 수정해야 합니다.

반대로 `api.ts` 한곳에서 관리하면 됩니다.

API 요청의 공통 관리 구조

화면 A · 화면 B · 화면 C

api.ts

백엔드 주소 + 로그인 토큰 + API 요청 관리

기존 백엔드 API

이런 구조를 **관심사의 분리(Separation of Concerns)**라고 합니다.

화면을 만드는 역할과 데이터를 가져오는 역할을 분리하는 것입니다.

④ src/components: 공통 화면 요소

버튼, 로딩 화면, 오류 화면처럼 여러 화면에서 반복해서 사용하는 요소를 모아 둡니다.

예를 들어 모든 화면에서 같은 모양의 버튼을 사용한다면 버튼을 매번 새로 만들 필요가 없습니다.

공통 버튼 하나를 만들어 여러 화면에서 불러다 쓰면 됩니다.

이렇게 하면 디자인을 변경할 때도 한곳만 수정하면 됩니다.

4. 앱을 켜서 AI 답변을 받을 때까지의 과정

이번 강의에서 가장 중요한 기술적인 내용입니다.

앱을 실행하고 메시지를 보내면 앱, 백엔드, Supabase가 서로 통신합니다.

처음에는 복잡해 보이지만 순서대로 살펴보면 간단합니다.

① 전체 작동 흐름

앱 실행부터 AI 답변까지

1. 앱 실행

   앱이 백엔드에 설정 정보를 요청합니다.

2. 백엔드 설정 수신

   `/api/config`에서 Supabase 주소, 공개 키, 캐릭터 선택지를 받아옵니다.

3. 익명 로그인

   Supabase에서 게스트 사용자를 생성하고 로그인 토큰을 받습니다.

4. 로그인 토큰 저장

   기기에 토큰을 저장해 앱을 다시 실행해도 기존 게스트로 이어집니다.

5. 사용자 데이터 요청

   토큰을 붙여 내 정보, 캐릭터, 대화 기록을 가져옵니다.

6. AI에게 메시지 전송

   `/api/chat`으로 메시지를 보냅니다.

7. AI 답변 수신

백엔드에서 생성한 답변을 여러 이벤트로 나누어 실시간 수신합니다.

② Supabase 익명 로그인은 무엇인가요?

Supabase SVG Icon - Free Supabase PNG & SVG Download

Supabase

사용자 인증과 데이터베이스 등의 기능을 제공하는 백엔드 서비스입니다.

이번 강의에서는 익명 로그인을 사용합니다.

익명 로그인은 이메일이나 비밀번호를 입력하지 않아도 임시 사용자 계정을 생성해 주는 기능입니다.

예를 들어 사용자가 AI 친구 앱을 처음 실행하면 다음과 같이 처리됩니다.

1. 앱이 Supabase에 익명 로그인을 요청합니다.
2. Supabase가 새로운 게스트 사용자 ID를 만듭니다.
3. Supabase가 로그인 토큰을 발급합니다.
4. 앱은 토큰을 스마트폰에 저장합니다.
5. 이후 백엔드에 요청할 때마다 토큰을 함께 보냅니다.

여기서 토큰은 일종의 신분증입니다.

백엔드는 이 신분증을 Supabase에 확인하고, 해당 사용자의 데이터만 제공할 수 있습니다.

중요한 점이 하나 있습니다.

같은 사람이 웹과 앱을 사용해도 각각 다른 익명 사용자가 생성됩니다.

웹 브라우저와 스마트폰은 각각 별도의 로그인 정보를 가지고 있기 때문입니다.

따라서 익명 로그인을 사용하는 동안에는 웹과 앱의 사용자 데이터가 자동으로 하나로 합쳐지지 않습니다.

5. AI 답변을 실시간으로 받는 스트리밍 기술

이번 강의에서 꼭 이해해야 할 개념입니다.

일반적인 API는 요청하면 결과를 한 번에 돌려줍니다.

하지만 AI는 답변을 만드는 데 시간이 걸립니다.

답변 전체가 완성될 때까지 기다렸다가 한꺼번에 보여 주면 사용자는 오래 기다리는 느낌을 받습니다.

그래서 이번 템플릿에서는 스트리밍(Streaming) 방식을 사용합니다.

① 일반 응답과 스트리밍 응답의 차이

일반적인 응답 방식

AI가 답변을 모두 만든 다음

완성된 답변 전체를 한 번에 전달

스트리밍 응답 방식

1

안녕하세요.

•

오늘은 어떤 일이

3

있으셨나요?

완료

답변 전송 종료

실제 데이터는 글자 조각이나 이벤트 단위로 전달될 수 있습니다.

스트리밍은 수도꼭지에서 물이 조금씩 흘러나오는 것과 비슷합니다.

물을 전부 모아 놓았다가 한꺼번에 붓는 것이 아니라, 생성되는 대로 사용자에게 전달합니다.

② 이번 앱에서 사용하는 이벤트 4가지

백엔드의 `/api/chat`은 답변을 한 번에 전달하지 않고 JSON 이벤트를 한 줄씩 전달합니다.

이벤트	의미	앱에서 하는 일
`delta`	답변 글자 조각	화면에 답변을 이어 붙임
`reminder`	약속 생성 정보	알림이나 약속 정보 처리
`memory`	기억 정리 결과	사용자 기억 정보 처리
`done`	답변 완료	스트리밍 종료 처리

예를 들어 AI가 다음과 같이 답변한다고 가정해 보겠습니다.

"내일 오전 9시에 운동하시는 건 어떠세요?"

앱은 `delta` 이벤트를 받아 글자를 화면에 계속 추가합니다.

AI가 약속을 생성하면 `reminder` 이벤트도 전달되고, 대화에서 중요한 내용을 기억으로 정리하면 `memory` 이벤트가 전달됩니다.

마지막으로 `done` 이벤트가 오면 답변 생성이 끝났다는 것을 알 수 있습니다.

③ 왜 Expo의 expo/fetch를 사용할까?

여기서 기술적인 문제가 하나 있습니다.

React Native의 기본 `fetch`는 이번 템플릿에서 필요한 스트리밍 응답 읽기 방식에 적합하지 않습니다.

그래서 Expo에서 제공하는 `expo/fetch`를 사용합니다.

이 도구를 이용하면 응답을 조금씩 읽어서 화면에 연결할 수 있습니다.

정리하면 다음과 같습니다.

- `fetch`: 서버에 데이터를 요청하는 기본 기능
- `expo/fetch`: Expo 환경에서 스트리밍 응답을 읽기 위해 사용하는 기능
- `delta`: 실시간으로 전달되는 답변 조각
- `done`: 답변 생성 완료 신호

이 차이를 이해하는 것이 중요합니다.

AI 서비스에서는 답변의 정확성만큼 사용자에게 답변을 전달하는 속도와 경험도 중요하기 때문입니다.

6. 앱에 넣으면 안 되는 비밀정보

보안 핵심

이번 강의에서 반드시 기억해야 할 내용입니다.

앱은 사용자의 스마트폰에 설치되는 프로그램입니다.

따라서 앱 파일을 다운로드한 사람이 내부 정보를 분석할 수 있습니다.

특히 `EXPO_PUBLIC_`으로 시작하는 환경변수는 앱을 빌드할 때 앱 파일 안에 포함됩니다.

즉, 공개 정보라고 생각해야 합니다.

① 공개해도 되는 정보

다음 정보는 앱에 넣을 수 있습니다.

- 백엔드 주소
- Supabase publishable key

이 정보는 서비스에 접근하기 위한 공개적인 식별 정보입니다.

단, 공개 키를 사용하더라도 실제 데이터 접근 권한은 인증과 데이터베이스 보안 정책으로 보호해야 합니다.

② 절대로 앱에 넣으면 안 되는 정보

다음 정보는 반드시 백엔드에만 보관해야 합니다.

- Supabase secret key
- LLM API 키
- 데이터베이스 비밀번호

왜 그럴까요?

예를 들어 LLM API 키를 앱에 넣었다고 가정해 보겠습니다.

누군가 앱 파일에서 이 키를 찾아내면 내 계정으로 AI API를 무단 호출할 수 있습니다.

그 결과 비용이 발생할 수 있습니다.

Supabase secret key가 노출되면 데이터 접근 권한이 악용될 위험도 있습니다.

올바른 비밀정보 관리 구조

모바일 앱

공개 가능

백엔드 주소, Supabase publishable key

백엔드

비밀정보 보관

LLM API 키, Supabase secret key, DB 비밀번호

앱은 백엔드 API를 통해서만 기능 사용

백엔드가 비밀정보를 사용해 외부 서비스 호출

여기서 중요한 설계 원칙은 다음과 같습니다.

비밀정보는 사용자에게 배포되는 앱이 아니라, 개발자가 통제하는 백엔드에 보관해야 합니다.

이 원칙은 모바일 앱뿐 아니라 웹 서비스, AI 에이전트, SaaS 서비스를 개발할 때도 동일하게 적용됩니다.

7. 객관식 퀴즈 풀이

Q. 앱의 `.env`에 `EXPO_PUBLIC_`으로 시작하는 이름으로 넣어도 되는 값은 무엇인가요?

1. Supabase secret key
2. LLM API 키
3. 백엔드 주소
4. 백엔드가 쓰는 데이터베이스 비밀번호

정답: 3번

해설

`EXPO_PUBLIC_`으로 시작하는 값은 앱 파일에 포함되어 외부에 공개될 수 있습니다.

따라서 공개해도 되는 백엔드 주소는 넣을 수 있지만, Supabase secret key, LLM API 키, 데이터베이스 비밀번호는 넣으면 안 됩니다.

앱은 백엔드 주소를 통해 요청하고, 실제 비밀정보를 사용하는 작업은 백엔드에서 처리해야 합니다.

8. 나의 프로젝트에 적용하기

이번 강의는 앞으로 진행할 프로젝트의 전체적인 아키텍처를 설계할 때 매우 중요합니다.

프로젝트	이번 강의의 적용 포인트
AI 친구 에이전트	웹과 모바일이 동일한 백엔드 API 사용, 스트리밍 답변 구현
MSM	일기·사진·음성 입력 화면과 AI 콘텐츠 생성 백엔드 분리
Travel Creator	모바일은 여행 일정·사진 입력, 백엔드는 일정 분석 및 추천
ASP 컨설팅	모바일은 고객 입력과 결과 표시, 백엔드는 진단 및 분석 처리

특히 MSM에 적용해 보겠습니다.

사용자가 스마트폰으로 일기를 작성하거나 사진을 올리면 앱은 입력 화면과 업로드를 담당합니다.

백엔드는 자료를 분석하고 사용자의 문체를 파악해 웹소설 형태로 변환합니다.

AI가 글을 생성하는 동안에는 스트리밍으로 결과를 보여 줄 수 있습니다.

이렇게 하면 모바일 앱은 복잡한 AI 처리 기능을 직접 구현하지 않아도 됩니다.

백엔드에 이미 만들어 놓은 AI 기능을 그대로 활용할 수 있기 때문입니다.

9. 반드시 기억해야 할 핵심 8가지

핵심 개념	반드시 기억할 내용
앱과 백엔드	앱은 화면, 백엔드는 데이터와 판단

핵심 개념	반드시 기억할 내용
Expo Router	파일 하나가 화면 하나이며 파일명이 주소가 됨
`_layout.tsx`	앱 전체의 공통 틀과 로그인 상태 관리
`api.ts`	백엔드 주소와 API 요청을 한곳에서 관리
Supabase 익명 로그인	게스트 사용자 생성 및 인증 토큰 발급
스트리밍	AI 답변을 생성되는 대로 조금씩 전달
`expo/fetch`	앱에서 스트리밍 응답을 읽는 데 사용
`EXPO_PUBLIC_`	앱에 포함되므로 공개 가능한 값만 저장

10. 오늘 강의 한 문장 요약

모바일 앱은 화면과 사용자 입력을 담당하고, 기존 백엔드는 인증·데이터·AI 판단을 담당하며, API와 스트리밍을 통해 두 시스템이 연결되는 구조를 이해하는 것이 이번 강의의 핵심입니다.

11. AI 서비스 기획자 관점에서 생각할 문제

이번 강의에서 가장 중요한 것은 단순히 파일 이름을 외우는 것이 아닙니다.

서비스를 설계할 때 다음 세 가지를 구분할 수 있어야 한다는 것입니다.

1. 사용자가 직접 보고 조작하는 기능은 어디에서 구현할 것인가?
2. 데이터와 AI 판단은 어디에서 처리할 것인가?
3. 보안이 필요한 정보는 어디에 보관할 것인가?

이 세 가지를 구분할 수 있다면 AI 코딩 에이전트에게도 훨씬 구체적으로 개발을 지시할 수 있습니다.

오늘의 핵심 교훈: 좋은 모바일 앱은 모든 기능을 앱 안에 만드는 것이 아니라, 앱과 백엔드가 각자 잘하는 일을 나누어 수행하도록 설계하는 것입니다.

Chapter07

챕터 07. 모바일 앱과 백엔드 연결 및 실기기 테스트

핵심 주제: 실제 스마트폰에서 앱 실행하기

1. 강의의 목적

이번 강의에서는 지금까지 준비한 모바일 앱을 실제 스마트폰에 연결하고 실행합니다.

앱을 개발하는 것과 실제 휴대폰에서 작동시키는 것은 다른 문제입니다. 이번에는 백엔드 주소를 설정하고, 개발 서버를 실행하고, QR 코드로 스마트폰에 앱을 연결하는 과정을 배웁니다.

오늘의 학습 목표는 다음과 같습니다.

1. `.env` 파일에 백엔드 주소를 설정하는 방법
2. Expo 개발 서버를 실행하고 스마트폰과 연결하는 방법
3. 실제 스마트폰에서 캐릭터 생성과 AI 대화를 테스트하는 방법
4. 앱과 웹에서 서로 다른 대화가 보이는 이유
5. 연결 오류가 발생했을 때 원인을 찾아 해결하는 방법

. 백엔드 주소 설정하기

① .env 파일이란?

`.env` 파일은 프로그램이 실행될 때 필요한 환경설정 값을 저장하는 파일입니다.

이번 모바일 앱에서는 가장 중요한 설정이 백엔드 주소입니다.

예를 들어 AI 친구 에이전트의 백엔드가 Google Cloud Run에 배포되어 있다면 다음과 같은 주소를 사용합니다.

```
EXPO_PUBLIC_API_URL=https://friend-agent-staging-xxxxxxxxxx-uc.a.run.app
```

여기서 주의할 점은 주소 끝에 `/`를 붙이지 않는 것입니다.

② .env 파일 만들기

프로젝트에는 기본 설정 예시인 `.env.example`이 있습니다.

이 파일을 복사해서 실제 설정 파일인 `.env`를 만듭니다.

운영체제	명령어
macOS / Linux	`cp .env.example .env`
Windows PowerShell	`Copy-Item .env.example .env`

그다음 `.env` 파일을 편집기로 열어 백엔드 주소를 입력합니다.

③ 백엔드가 정상적으로 작동하는지 먼저 확인

앱에 주소를 입력하기 전에 브라우저에서 다음 주소를 열어 봅니다.

```
<백엔드 주소>/health
```

정상적으로 작동한다면 다음과 같은 결과가 나옵니다.

```
{"status":"ok"}
```

이것은 백엔드 서버가 정상적으로 응답하고 있다는 의미입니다.

중요한 원칙은 앱에서 문제가 발생했을 때 앱부터 의심하지 말고 백엔드가 정상적으로 작동하는지 먼저 확인하는 것입니다.

④ .env 파일은 GitHub에 올리지 않습니다.

`.env`에는 실행 환경에 따라 달라지는 설정값이 들어갑니다.

이 파일은 `.gitignore`에 등록되어 있어 Git에 커밋되지 않도록 관리합니다.

이번 강의에서 백엔드 주소는 공개 가능한 값이지만, 실제 프로젝트에서는 API 키 등 민감한 값이 실수로 포함되지 않도록 주의해야 합니다.

3. Expo 개발 서버 실행하기

① 개발 서버란?

개발 서버는 노트북에서 작성한 모바일 앱을 스마트폰에서 실행할 수 있도록 연결해 주는 프로그램입니다.

명령 프롬프트 또는 터미널에서 다음 명령을 실행합니다.

```
npx expo start
```

그러면 터미널에 QR 코드가 나타납니다.

Expo 실행과 스마트폰 연결

노트북

Expo 개발 서버 실행

스마트폰

Expo Go 실행

QR 코드 스캔

노트북에서 앱 번들을 스마트폰으로 전달

Expo Go에서 모바일 앱 실행

② 스마트폰에서 실행하는 방법

1. Android 스마트폰에서 Expo Go 앱을 실행합니다.
2. `Scan QR code`를 선택합니다.
3. 노트북 터미널에 나타난 QR 코드를 찍습니다.
4. 스마트폰이 노트북에서 앱 번들을 다운로드합니다.
5. 다운로드가 끝나면 모바일 앱이 실행됩니다.

여기서 중요한 조건이 있습니다.

노트북과 스마트폰은 같은 와이파이에 연결되어 있어야 합니다.

Windows에서 처음 실행하면 Node.js의 네트워크 접근을 허용할 것인지 묻는 방화벽 창이 나올 수 있습니다. 이때 개인 네트워크 접근을 허용해야 스마트폰이 노트북에 연결될 수 있습니다.

③ 연결이 안 될 때 사용하는 명령어

명령어	사용 목적
`npx expo start`	기본 개발 서버 실행
`npx expo start --tunnel`	와이파이 또는 방화벽 문제로 연결이 안 될 때
`npx expo start --clear`	캐시를 삭제하고 개발 서버 재실행

특히 `--tunnel`은 같은 와이파이에서 기기끼리 통신할 수 없는 환경에서 유용합니다.

다만 외부 터널 서비스를 거치기 때문에 속도가 느려질 수 있고, 터널 주소가 인터넷에 공개될 수 있다는 점을 기억해야 합니다.

4. 실제 스마트폰에서 확인할 사항

QR 코드로 앱을 실행했다면 이제 실제 사용자의 입장에서 테스트합니다.

다음 네 가지를 확인해야 합니다.

스마트폰 테스트 체크리스트

Frontiers | Visual identities and phygital branding in video games: symbolic consumption strategies among Spanish Generation Z

1. 캐릭터 생성

처음 실행했을 때 캐릭터 선택 화면이 나타나는지, 이름과 모습, 말투를 설정할 수 있는지 확인합니다.

MyCompanion | complex ptsd

- . 대화 화면

캐릭터 이름과 아바타가 보이는지, 메시지를 보내면 답변이 표시되는지 확인합니다.

Sovereign AI for all. FREE on iOS & Android.

3. 답변 스트리밍

AI 답변이 완성된 뒤 한 번에 나타나는 것이 아니라 글자 단위로 이어서 나타나는지 확인합니다.

The App Bros: Tailored AI (Artificial Intelligence) App Development Services

4. 앱 재실행

Expo Go를 완전히 종료했다가 다시 실행해도 기존 캐릭터와 대화가 유지되는지 확인합니다.

이 네 가지가 모두 정상적으로 작동한다면 앱과 백엔드의 기본 연결이 성공한 것입니다.

5. 같은 백엔드인데 웹과 앱의 대화가 다른 이유

오늘의 핵심 개념

이번 강의에서 가장 중요한 이해 점검 부분입니다.

스마트폰 앱에서 캐릭터를 만들고 대화를 했습니다. 그런데 같은 백엔드를 사용하는 노트북 브라우저에서는 캐릭터가 없고 대화도 보이지 않습니다.

데이터가 사라진 것일까요?

아닙니다. 웹과 앱이 서로 다른 게스트 사용자로 로그인했기 때문입니다.

① Supabase는 사용자를 구분합니다.

앞선 챕터에서 배운 Supabase 익명 로그인을 떠올려 보겠습니다.

앱과 웹 브라우저는 각각 독립적으로 익명 로그인을 수행합니다.

같은 사람, 서로 다른 게스트 ID

스마트폰 앱

게스트 A

캐릭터와 대화 A 저장

노트북 웹

게스트 B

별도의 사용자로 로그인

동일한 백엔드

사용자 ID를 기준으로 데이터를 구분해서 조회

Supabase 데이터베이스

게스트 A의 데이터와 게스트 B의 데이터를 별도로 저장

백엔드는 데이터를 조회할 때 로그인한 사용자의 ID를 기준으로 필터링합니다.

따라서 앱에서 생성한 캐릭터와 대화는 게스트 A의 데이터이고, 브라우저에서 로그인한 사용자는 게스트 B이므로 앱의 대화를 볼 수 없습니다.

② 데이터가 실제로 저장되었는지 확인하는 방법

Supabase의 Table Editor에서 `messages` 테이블을 확인합니다.

앱에서 방금 보낸 메시지가 테이블에 저장되어 있다면 데이터가 사라진 것이 아니라 정상적으로 저장된 것입니다.

확인할 내용은 다음과 같습니다.

- 앱에서 보낸 메시지가 `messages` 테이블에 존재하는가?
- 해당 메시지의 사용자 ID가 앱의 익명 사용자 ID와 일치하는가?
- 브라우저에서는 다른 사용자 ID로 조회하고 있는가?

이 세 가지를 확인하면 문제의 원인을 이해할 수 있습니다.

③ 웹과 앱에서 같은 대화를 보려면?

사용자가 어떤 기기를 사용하든 동일한 사람이라는 것을 알아볼 수 있어야 합니다.

대표적인 방법이 구글 로그인과 같은 정식 사용자 인증입니다.

예를 들어 다음과 같은 구조입니다.

- 스마트폰에서 구글 로그인

- 노트북에서도 같은 구글 계정으로 로그인
- Supabase가 동일한 사용자로 인식
- 백엔드가 동일한 사용자 ID의 대화 기록 제공

이렇게 하면 기기가 달라도 같은 캐릭터와 대화를 이어갈 수 있습니다.

이것은 향후 AI 친구 에이전트를 실제 서비스로 운영할 때 반드시 고려해야 할 기능입니다.

6. 첫 번째 답변이 느린 이유: 콜드 스타트

앱에서 처음 메시지를 보냈을 때 답변이 유독 늦게 도착할 수 있습니다.

이때 생각해야 할 개념이 **콜드 스타트(Cold Start)**입니다.

Google Cloud Run과 같은 서버리스 환경은 요청이 없으면 인스턴스를 줄이거나 종료할 수 있습니다.

오랫동안 요청이 없다가 새로운 요청이 들어오면 서버 인스턴스를 다시 시작해야 합니다.

이 과정에서 시간이 추가로 걸립니다.

상황	설명
첫 번째 요청	쉬고 있던 인스턴스를 깨우는 시간이 추가될 수 있음
두 번째 요청	이미 실행 중인 인스턴스를 사용하므로 상대적으로 빠를 수 있음

쉽게 비유하면 영업시간 전에 문을 닫아 둔 식당에 첫 손님이 왔을 때 주방을 준비하는 시간이 필요한 것과 같습니다.

이 개념은 향후 서비스 운영 비용과 응답 속도를 함께 설계할 때 중요합니다.

7. 연결이 안 될 때 문제 해결하기

앱 개발에서 중요한 능력 중 하나는 오류가 발생했을 때 원인을 단계적으로 좁혀 가는 것입니다.

증상	확인할 사항 및 해결 방법
QR 코드를 찍어도 연결 실패	같은 와이파이인지 확인, Windows 방화벽에서 Node.js 허용
터널 실행 시 ngrok 오류	안내에 따라 `@expo/ngrok` 설치
Expo Go 버전 호환 오류	프로젝트 SDK와 Expo Go 지원 버전 확인, Expo Go 업데이트
백엔드 주소가 비어 있음	`.env`의 `EXPO_PUBLIC_API_URL` 확인 후 `--clear` 실행
익명 로그인 실패	Supabase의 Allow anonymous sign-ins 설정 확인
`/api/config` 요청 실패	백엔드 주소 확인, `/health` 응답 확인

여기서 중요한 것은 오류가 발생했다고 무조건 코드를 수정하지 않는 것입니다.

다음 순서로 확인하면 원인을 찾기가 쉽습니다.

1. 백엔드가 정상적으로 작동하는가?
2. `.env`에 올바른 주소가 입력되었는가?
3. 노트북과 스마트폰이 통신할 수 있는가?
4. Supabase 익명 로그인이 가능한가?
5. 앱에서 API 요청이 정상적으로 전달되는가?

8. 나의 프로젝트에 적용하기

이번 강의는 앞으로 개발할 AI 서비스의 실제 운영 환경을 이해하는 데 도움이 됩니다.

프로젝트	적용할 내용
AI 친구 에이전트	모바일 앱에서 캐릭터 생성, 대화, 기록 저장 및 재실행 테스트
MSM	모바일에서 일기와 사진을 입력하고 서버에서 웹소설 생성
Travel Creator	여행 중 스마트폰으로 사진과 장소를 입력하고 서버에서 일정 생성
ASP 컨설팅	고객이 모바일에서 진단 내용을 입력하고 백엔드에서 분석 결과 제공

특히 MSM을 생각해 보겠습니다.

사용자가 스마트폰으로 일기를 입력하고 사진을 첨부한 뒤, AI가 이를 분석해 웹소설을 만들어 준다고 가정하겠습니다.

이때 스마트폰을 바꾸거나 노트북에서 접속해도 동일한 작업 결과를 확인할 수 있어야 합니다.

이를 위해서는 익명 로그인만으로는 부족할 수 있고, 정식 계정 인증과 사용자별 데이터 동기화가 필요합니다.

따라서 이번 강의의 익명 로그인은 초기 테스트에는 적합하지만, 실제 서비스 출시 단계에서는 계정 통합과 사용자 데이터 이전까지 설계해야 합니다.

9. 반드시 기억해야 할 핵심 8가지

개념	핵심 내용
`.env`	백엔드 주소 등 실행 환경 설정
`/health`	백엔드가 정상 작동하는지 확인하는 주소
`npx expo start`	모바일 앱 개발 서버 실행

개념	핵심 내용
QR 코드	Expo Go와 노트북 개발 서버 연결
`--tunnel`	네트워크 연결이 제한된 환경에서 우회 연결
Supabase 익명 로그인	기기별 게스트 사용자 생성
사용자 ID	사용자의 데이터 접근 범위를 구분하는 기준
콜드 스타트	쉬고 있던 서버 인스턴스가 다시 시작되며 발생하는 지연

10. 주관식 퀴즈 풀이

Q. 앱에서 캐릭터를 만들고 대화한 뒤, 같은 백엔드를 노트북 브라우저로 열었더니 캐릭터 만들기 화면부터 나오고 대화도 없었습니다. 데이터가 사라진 것이 아니라는 것을 어떻게 확인할 수 있는지, 왜 이렇게 보이는지 설명해 보세요.

모범 답안

앱에서 나눈 대화가 사라진 것이 아니라, 스마트폰 앱과 노트북 브라우저가 서로 다른 익명 사용자로 로그인했기 때문입니다.

데이터가 정상적으로 저장되었는지 확인하려면 Supabase의 Table Editor에서 `messages` 테이블을 열어 앱에서 보낸 메시지가 저장되어 있는지 확인하면 됩니다. 그리고 해당 메시지의 사용자 ID를 확인하면 앱의 게스트 계정에 연결되어 있다는 사실을 알 수 있습니다.

백엔드는 로그인한 사용자의 ID를 기준으로 데이터를 조회하기 때문에, 노트북 브라우저에서 다른 게스트로 로그인하면 앱의 대화가 보이지 않습니다.

두 기기에서 같은 대화를 이어가려면 구글 로그인과 같은 공통 인증 방식을 도입해 두 기기가 동일한 사용자 ID로 인식되도록 해야 합니다.

11. 오늘 강의 한 문장 요약

모바일 앱은 `.env`에 백엔드 주소를 설정하고 Expo 개발 서버와 QR 코드로 실제 스마트폰에 연결하며, Supabase의 사용자 ID를 기준으로 데이터를 저장하고 조회한다는 것을 이해하는 것이 이번 강의의 핵심입니다.

12. AI 서비스 개발자로서 얻어야 할 교훈

이번 강의에서 가장 중요한 것은 앱이 실행되는 것 자체보다 앱과 서버가 실제 사용자 환경에서 어떻게 연결되고, 데이터가 누구에게 귀속되며, 문제가 생겼을 때 어디서 원인을 찾아야 하는지를 이해하는 것입니다.

AI 서비스는 단순히 AI가 답변을 잘하는 것만으로 완성되지 않습니다.

- 스마트폰에서 정상적으로 실행되어야 합니다.

- 사용자의 데이터가 안전하게 저장되어야 합니다.
- 여러 기기에서 같은 사용자 경험을 제공해야 합니다.
- 첫 응답 속도와 네트워크 문제도 관리해야 합니다.

이것이 AI 기능을 만드는 단계에서 실제로 운영할 수 있는 서비스로 발전시키는 과정입니다.

Chapter08

챕터 08. 모바일 앱 화면 수정과 사진으로 약속 만들기

핵심 주제: 실시간 화면 수정과 스마트폰 카메라 활용

1. 강의의 목적

이번 강의에서는 지금까지 만든 모바일 앱에 실제 사용자에게 필요한 기능을 추가합니다.

크게 두 가지 작업을 합니다.

1. 코드를 수정하고 저장하면 스마트폰 화면이 즉시 바뀌는 것을 확인합니다.
2. 스마트폰 카메라나 사진첩으로 일정표를 가져와 글자를 읽고, AI 친구에게 약속 생성을 요청하는 기능을 만듭니다.

이번 강의의 중요한 원칙은 앱은 스마트폰의 기능을 활용하고, 복잡한 AI 처리와 데이터 판단은 기존 백엔드에 맡긴다는 것입니다.

. 코드를 저장하면 스마트폰 화면이 바로 바뀌는 이유

① Expo의 빠른 화면 반영

지난 강의에서 `npx expo start`를 실행하고 QR 코드로 스마트폰을 연결했습니다.

이 상태에서 노트북의 코드를 수정하고 저장하면, 몇 초 안에 스마트폰 화면이 바뀝니다.

예를 들어 `src/app/index.tsx` 파일에서 다음 문장을 찾아보겠습니다.

```
첫 메시지를 보내 보세요.
```

이 문장을 원하는 문장으로 바꾸고 저장하면 스마트폰 화면에도 변경 내용이 반영됩니다.

왜 앱을 다시 설치하지 않아도 될까요?

Expo Go가 개발 서버에서 변경된 JavaScript 번들을 다시 받아 화면을 새로 그리기 때문입니다.

코드 수정부터 화면 반영까지

1. 노트북에서 코드 수정
2. 파일 저장
3. Expo 개발 서버가 변경사항 감지
4. 스마트폰의 Expo Go가 변경된 번들을 반영
5. 앱 화면 변경 확인

앱 재설치 없이 반영

이 기능을 개발 중 빠른 반복 작업에 활용할 수 있습니다.

② 일부러 오류를 만들어 보기

이번에는 괄호 하나를 일부러 지우고 저장해 봅니다.

그러면 스마트폰에 빨간 오류 화면이 나타나고, 어느 파일의 몇 번째 줄에서 문제가 발생했는지 표시됩니다.

괄호를 원래대로 복구하고 저장하면 오류 화면이 사라집니다.

여기서 중요한 학습 포인트가 있습니다.

코딩 에이전트가 코드를 수정한 뒤 앱에 오류가 발생하면, 빨간 오류 화면의 내용을 그대로 에이전트에게 전달하면 됩니다.

오류를 추측해서 설명하는 것보다 실제 오류 메시지를 전달하는 편이 문제 해결에 유리합니다.

3. 화면을 수정하기 전에 데이터 구조부터 정하기

이번에는 웹 화면에는 있지만 모바일 앱에는 없는 정보를 추가합니다.

웹 화면에는 AI 답변 아래에 다음과 같은 메모가 표시됩니다.

- 기억했어요
- 약속 알림

그런데 현재 모바일 앱은 AI의 답변 글자만 보여 줍니다.

이 기능을 추가하려면 먼저 세 가지를 결정해야 합니다.

1. 어떤 정보를 보여 줄 것인가?

2. 그 정보는 백엔드의 어디에서 가져올 것인가?
3. 어느 파일을 수정해야 하는가?

① 추가할 기능 세 가지

기능	데이터 출처	수정 파일
A. 기억·약속 메모	`/api/chat`의 `memory`, `reminder` 이벤트	`api.ts`, `index.tsx`
B. 메시지 보낸 시각	`/api/messages`의 `created_at` 등	`api.ts`, `index.tsx`
C. 캐릭터 관심사	백엔드 캐릭터의 `interests`	`character.tsx`

이번 강의에서는 A를 먼저 구현하고, B와 C는 사진 기능까지 끝낸 뒤 시간이 남으면 진행합니다.

중요한 점은 세 가지 기능 모두 백엔드를 수정하지 않는다는 것입니다.

백엔드가 이미 필요한 데이터를 보내 주고 있기 때문에 앱 화면만 수정하면 됩니다.

② 왜 데이터 출처를 먼저 확인해야 할까요?

예를 들어 앱에 사용자의 생일을 보여 주고 싶은데 백엔드가 생일 정보를 보내 주지 않는다고 가정하겠습니다.

이때 앱 코드만 계속 수정해도 생일 정보를 만들어 낼 수는 없습니다.

먼저 백엔드에서 해당 데이터를 제공하도록 설계해야 합니다.

즉, 화면을 만들기 전에 필요한 데이터가 어디에 있고, 어떤 형태로 전달되는지 확인하는 것이 먼저입니다.

이것이 AI 코딩 에이전트에게 일을 맡기기 전에 기획자가 해야 하는 중요한 작업입니다.

4. A 기능: AI 답변 아래 기억과 약속 메모 표시

① 어떤 기능인가요?

AI 친구와 대화하다 보면 사용자의 중요한 정보를 기억하거나 약속을 저장할 수 있습니다.

예를 들어 다음과 같은 대화를 생각해 보겠습니다.

- 사용자: "나 고양이 두 마리 키워."
- AI: "고양이 두 마리를 키우시는군요!"
- 답변 아래: "🧠 기억했어요: 고양이 두 마리를 키움"

또 다른 예입니다.

- 사용자: "내일 아침 8시에 물 마시라고 알려 줘."
- AI: "내일 아침 8시에 알려 드릴게요."
- 답변 아래: "⏰ 내일 오전 8시 물 마시기"

② 데이터는 어디서 가져오나요?

백엔드의 `/api/chat`이 답변을 스트리밍할 때 다음 이벤트를 함께 전달합니다.

이벤트	전달 내용	화면 표시
`memory`	새롭게 기억한 정보	🧠 기억했어요
`reminder`	저장된 약속 정보	⏰ 약속 알림

따라서 앱은 이벤트를 받아서 AI 답변 아래에 회색 작은 글씨로 표시하면 됩니다.

모바일 대화 화면 예시

AI 친구

안녕하세요! 고양이 두 마리를 키우시는군요. 정말 귀엽겠어요.

🧠 기억했어요: 고양이 두 마리를 키움

AI 친구

내일 아침 8시에 물을 마시도록 알려 드릴게요.

⏰ 내일 오전 8시 물 마시기

③ 왜 지난 대화에는 메모가 보이지 않을까요?

여기서 중요한 차이가 있습니다.

기억과 약속 메모는 메시지 자체에 저장된 내용이 아니라, 해당 대화 차례에서 전달된 스트리밍 이벤트입니다.

따라서 앱을 닫았다 다시 열면 과거 메시지는 불러오지만, 과거에 전달되었던 `memory`, `reminder` 이벤트까지 다시 불러오지는 않습니다.

지난 대화에도 메모를 표시하려면 백엔드가 해당 메모를 메시지와 함께 저장하고, 대화 기록을 조회할 때 다시 전달하도록 바꿔야 합니다.

이 사례는 앱 화면에 무엇을 보여 줄 것인지뿐 아니라, 그 정보가 언제 생성되고 어디에 저장되는지도 알아야 한다는 점을 보여 줍니다.

④ A 기능 객관식 퀴즈

Q. 앱을 닫았다 다시 열었더니 지난 대화의 기억 메모가 사라졌습니다. 이유는 무엇일까요?

정답: 2번

`memory` 메모는 해당 대화 차례의 `/api/chat` 스트리밍 이벤트로만 전달되고, 과거 대화를 가져오는 `/api/messages`에는 포함되어 있지 않기 때문입니다.

앱이 기억을 삭제한 것이 아닙니다. 애초에 과거 메시지와 함께 메모를 저장하고 다시 전달하는 구조가 아니었던 것입니다.

5. 사진으로 약속 만들기: 이번 강의의 핵심 기능

이번 강의에서 가장 실용적인 기능입니다.

스마트폰으로 일정표나 안내문을 촬영하면 사진 속 글자를 읽고, 그 내용을 AI 친구에게 전달해 약속을 만들 수 있도록 합니다.

예를 들어 종이에 다음과 같이 적혀 있다고 가정하겠습니다.

```
11월 20일 오후 3시 치과
```

앱에서 사진을 찍으면 글자를 읽어 입력창에 넣어 줍니다. 사용자가 내용을 확인하고 전송하면 AI가 약속을 생성합니다.

① OCR이란?

Schichtplan zu Kalender: Dienstpläne und Schichten digitalisieren | Smart Calendars AI | Smart Calendars AI

OCR (Optical Character Recognition)

사진이나 이미지 안에 있는 글자를 컴퓨터가 읽을 수 있는 텍스트로 바꾸는 기술입니다.

쉽게 말하면 사진 속 글자를 사람이 직접 타이핑하지 않아도 컴퓨터가 대신 읽어 주는 기술입니다.

이번 강의에서는 OCR 기능을 새로 개발하지 않습니다. 기존 백엔드에 이미 구현된 OCR API를 모바일 앱에서 호출합니다.

② 백엔드에 이미 준비된 기능

API 또는 설정	역할
`POST /api/ocr`	사진을 받아 글자를 읽음
`photo`	사진 파일을 전달하는 필드 이름
`ocr_enabled`	백엔드에서 OCR 기능을 켰는지 확인
`OCR_ENABLED=true`	백엔드 OCR 기능 활성화 설정

사진을 보내면 백엔드는 다음과 같은 결과를 반환합니다.

```
{
  "text": "11월 20일 오후 3시 치과"
}
```

앱은 이 글을 입력창에 넣어 주고, 사용자가 확인한 뒤 AI에게 전송합니다.

③ 먼저 OCR이 켜져 있는지 확인합니다.

브라우저에서 백엔드의 다음 주소를 엽니다.

```
<백엔드 주소>/api/config
```

응답에 다음과 같은 값이 있는지 확인합니다.

```
{
  "ocr_enabled": true
}
```

`false`라면 백엔드의 OCR 기능이 꺼져 있다는 뜻입니다.

이 경우 OCR 프로그램인 Tesseract가 배포 이미지에 포함되어 있는지, `OCR_ENABLED` 환경 변수가 설정되어 있는지 확인해야 합니다.

백엔드 기능이 꺼져 있다면 앱 화면만 수정해서는 해결할 수 없습니다.

6. 스마트폰에서 사진을 가져오는 두 가지 방법

이번 강의에서는 사진을 가져오는 방법으로 두 가지 Expo 라이브러리를 사용합니다.

Meituan Deleted Users' Photos: Overbroad Permissions Are Worse Than a Privacy Leak · Vonng

① expo-image-picker

스마트폰의 사진첩을 열어 이미 촬영해 둔 사진을 선택합니다.

- 일정표 사진
- 화면 캡처
- 저장된 안내문 이미지

Post — TechSesh

② expo-camera

앱 안에 카메라 화면을 직접 띄워 사진을 촬영합니다.

- 종이 일정표를 바로 촬영
- 촬영 안내 문구 표시
- 촬영 버튼과 닫기 버튼 제공

두 라이브러리 모두 카메라나 사진첩에 접근하므로 사용자 권한이 필요합니다.

또한 강의 기준 SDK 58에서는 두 라이브러리 모두 Expo Go에서 사용할 수 있습니다.

주의할 점도 있습니다.

- Expo Go에서 권한을 요청하면 Expo Go의 이름과 안내 문구가 표시됩니다.
- `app.json`에 작성한 내 앱의 권한 설명은 실제 빌드한 앱에서 확인할 수 있습니다.
- `expo-camera`의 실제 카메라 미리보기는 스마트폰에서 확인해야 합니다. 에뮬레이터나 시뮬레이터에서는 표시되지 않을 수 있습니다.

7. 사진으로 약속 만들기의 전체 구조

이제 앱에서 어떤 기능을 만들고 각 기능이 무엇을 담당하는지 정리해 보겠습니다.

사진 촬영부터 약속 생성까지

1. 대화 화면에서 📷 버튼 선택

   OCR이 활성화되어 있을 때만 버튼을 보여 줍니다.

2. 사진 가져오기 방식 선택

   사진첩에서 고르기 또는 카메라로 찍기

3. 사진 선택 또는 촬영

   사진첩은 `expo-image-picker`, 카메라는 `expo-camera` 사용

4. 백엔드 OCR 요청

   로그인 토큰과 사진 파일을 `/api/ocr`로 전송

5. 읽은 글을 입력창에 표시

   OCR 결과 앞에 약속 생성을 요청하는 안내 문장을 추가

6. 사용자가 확인하고 수정

   날짜와 시간 등이 정확한지 확인한 뒤 전송

7. AI가 약속 생성

   기존 백엔드가 약속을 만들고 결과를 전달

앱에서 수정할 파일과 기능을 정리하면 다음과 같습니다.

구성 요소	역할
대화 화면의 📷 버튼	사진 기능 진입
`expo-image-picker`	사진첩에서 사진 선택
`src/app/camera.tsx`	카메라 미리보기와 촬영
`src/lib/api.ts`	로그인 토큰을 붙여 OCR API 호출
입력창	읽은 글을 보여 주고 수정할 수 있도록 제공

① 왜 OCR 결과를 바로 보내지 않을까요?

이 부분은 서비스 기획 관점에서도 매우 중요합니다.

OCR은 사진 속 글자를 항상 정확하게 읽지는 못합니다.

- 손글씨가 흘려 쓰인 경우
- 사진이 기울어진 경우
- 조명이 어두운 경우
- 글자가 작거나 흐릿한 경우

특히 날짜나 시간이 잘못 인식되면 사용자가 엉뚱한 시간에 알림을 받게 됩니다.

따라서 OCR 결과를 바로 약속으로 저장하지 않고, 사람이 한 번 확인하는 단계를 둡니다.

이것을 HITL(Human-in-the-Loop), 즉 사람이 중간에 확인하는 구조라고 이해하시면 됩니다.

AI가 초안을 만들고 사람이 검토한 다음 최종 실행하는 방식입니다.

② 코딩 에이전트에게 맡길 때

코딩 에이전트에게는 다음과 같이 요청합니다.

> 일정표를 카메라로 찍거나 사진첩에서 골라서, 사진 속 글자를 읽어 입력창에 넣어 주는 기능을 만들어 줘. 글자 읽기는 백엔드의 `/api/ocr`을 사용하면 돼.

중요한 것은 단순히 "사진 기능을 만들어 줘"라고 하는 것이 아니라, 기존 백엔드 API와 앱의 역할을 명확히 알려 주는 것입니다.

③ 실제 스마트폰에서 확인할 사항

1. 종이에 `11월 20일 오후 3시 치과`라고 크게 적습니다.
2. 앱에서 📷 버튼을 누르고 카메라 촬영을 선택합니다.
3. 카메라 권한을 허용하고 종이를 촬영합니다.
4. 읽은 글이 입력창에 들어왔는지 확인합니다.
5. 잘못 읽힌 부분이 있다면 직접 수정합니다.
6. 전송한 뒤 AI가 약속을 생성하는지 확인합니다.
7. 이번에는 사진첩에서 사진을 선택해 동일하게 테스트합니다.
8. 카메라 권한을 거부했을 때 앱이 멈추지 않고 안내하는지도 확인합니다.

8. OCR 기능이 실패할 때 원인 찾기

사진을 보냈는데 오류가 발생한다면 HTTP 상태 코드를 먼저 확인합니다.

오류	원인	해결 방향
404	백엔드 OCR 기능이 꺼져 있음	`ocr_enabled`, `OCR_ENABLED` 확인
401	로그인 토큰이 전달되지 않음	인증 헤더 확인
400	사진 파일이 없거나 전달 방식이 잘못됨	`photo` 필드 확인
글자가 엉뚱하게 인식됨	사진 품질 문제	밝은 곳에서 글자를 크게 촬영

여기서 특히 주의해야 할 기술적인 내용이 있습니다.

multipart와 Content-Type

일반적인 API 요청에서는 JSON 형식으로 데이터를 보낼 수 있습니다.

하지만 사진 파일을 보낼 때는 `multipart/form-data`라는 방식을 사용합니다.

이 방식은 사진 파일과 기타 데이터를 하나의 요청에 담아 전달합니다.

템플릿의 공통 API 헤더에 JSON용 `Content-Type`이 설정되어 있다면 OCR 요청에서는 이를 그대로 사용하면 안 됩니다.

multipart 요청의 `Content-Type`에는 파일과 데이터의 경계를 나타내는 값이 필요하기 때문입니다.

따라서 사진을 보낼 때는 요청이 적절한 multipart 헤더를 만들도록 해야 합니다. 일반적으로 `FormData`를 사용하는 경우 `Content-Type`을 직접 지정하지 않고 요청 환경이 경계값을 포함해 설정하도록 둡니다.

이것은 텍스트를 보내는 API와 사진을 보내는 API가 데이터 전달 방식에서 다르다는 중요한 사례입니다.

④ 사진 기능 객관식 퀴즈

Q. 다음 중 앱 코드를 수정해야 하는 경우는 무엇인가요?

정답: 3번

선택지	앱 코드 수정 여부	이유
1. OCR 엔진을 더 정확하게 변경	불필요	백엔드에서 처리
2. 읽은 글로 약속을 만드는 방식 변경	불필요	백엔드의 AI 처리 로직
3. 카메라 화면의 안내 문구 변경	필요	앱 화면에 표시되는 내용
4. 백엔드에서 OCR 끄기	불필요	백엔드 설정 변경

이 문제의 핵심은 어떤 기능을 변경하느냐에 따라 앱을 수정할지 백엔드를 수정할지 구분하는 것입니다.

9. 코딩 에이전트와 일할 때 확인할 사항

이번 강의에서는 코딩 에이전트에게 기능 구현을 맡기지만, 에이전트가 작성한 코드를 무조건 신뢰해서는 안 됩니다.

특히 새로운 패키지를 설치하겠다고 할 때는 다음 두 가지를 확인합니다.

① Expo SDK와 호환되는 패키지인가?

일반적인 설치 명령보다 다음 명령을 사용하는 것이 좋습니다.

```
npx expo install 패키지명
```

Expo SDK에 맞는 버전을 선택하는 데 도움이 됩니다.

② Expo Go에서 실행할 수 있는가?

네이티브 코드가 포함된 패키지는 Expo Go에서 실행되지 않을 수 있습니다.

이런 패키지를 사용하려면 개발용 빌드가 필요할 수 있으므로, 현재 강의의 개발 흐름에 영향을 줍니다.

작업 완료 후 반드시 실행할 검사

```
# TypeScript 타입 검사
npx tsc --noEmit
```

```
# Expo 의존성과 설정 검사
npx expo-doctor
```

명령어	확인하는 내용
`tsc --noEmit`	TypeScript 코드의 타입 오류
`expo-doctor`	Expo 의존성 및 프로젝트 설정 문제

두 명령어에서 문제가 발견되면 에이전트에게 오류 내용을 전달해 수정하도록 합니다.

10. 나의 프로젝트에 적용하기

이번 강의에서 배운 기술은 특히 MSM과 Travel Creator에 직접 활용할 수 있습니다.

프로젝트	적용 아이디어
AI 친구 에이전트	대화 중 기억과 약속 메모 표시, 일정표 사진으로 약속 생성
MSM	일기장이나 손글씨 메모를 촬영해 원본 이야기로 변환
Travel Creator	항공권·호텔 예약 확인서 사진에서 일정과 예약 정보 추출
ASP 컨설팅	명함, 회의자료, 현장 점검표를 촬영해 상담 자료로 변환

특히 Travel Creator에서는 사진 OCR 기능을 확장할 가능성이 있습니다.

예를 들어 여행자가 항공권이나 호텔 예약 확인서를 촬영하면 날짜, 출발 시간, 숙박 기간 등을 추출해 여행 일정에 반영할 수 있습니다.

MSM에서는 손으로 쓴 일기나 오래된 메모를 디지털 콘텐츠로 바꾸는 데 활용할 수 있습니다.

두 프로젝트 모두 사진을 입력 수단으로 활용하고, AI가 내용을 분석하는 구조이므로 이번 강의의 설계 원칙을 그대로 적용할 수 있습니다.

11. 반드시 기억해야 할 핵심 10가지

핵심 개념	기억할 내용
Expo 빠른 반영	코드를 저장하면 앱 재설치 없이 화면 변경 가능
오류 화면	파일과 줄 번호를 확인해 에이전트에게 전달
데이터 출처	화면을 수정하기 전에 필요한 데이터가 어디서 오는지 확인
`memory` 이벤트	새롭게 기억한 정보를 전달
`reminder` 이벤트	새로 생성된 약속 정보를 전달
`expo-image-picker`	사진첩에서 이미지 선택

핵심 개념	기억할 내용
`expo-camera`	앱 안에서 직접 사진 촬영
OCR	이미지 속 글자를 텍스트로 변환
HITL	AI 결과를 사람이 확인한 뒤 실행
타입·환경 검사	`tsc --noEmit`, `expo-doctor` 실행

12. 오늘 강의 한 문장 요약

Expo의 빠른 화면 반영 기능을 활용해 모바일 앱을 개선하고, 카메라와 사진첩으로 가져온 이미지를 백엔드 OCR에 전달해 사람이 확인한 뒤 AI가 약속을 생성하도록 구현하는 것이 이번 강의의 핵심입니다.

13. AI 서비스 개발자로서 얻어야 할 교훈

이번 강의에서 배운 가장 중요한 것은 기능을 구현하기 전에 역할과 데이터 흐름을 먼저 설계하는 것입니다.

사진으로 약속을 만드는 기능을 예로 들면 다음과 같습니다.

- 스마트폰: 사진 촬영과 선택
- 앱: 사진 전달, OCR 결과 표시, 사용자 확인
- 백엔드: OCR 처리와 약속 생성
- AI: 읽은 내용의 의미를 이해하고 약속 처리

이렇게 역할을 나누면 OCR 엔진을 교체하거나 약속 생성 로직을 개선해도 모바일 앱을 다시 만들 필요가 없습니다.

반대로 카메라 화면이나 입력 방식이 바뀌면 앱을 수정해야 합니다.

오늘의 핵심 교훈은 AI 코딩 에이전트에게 코드를 맡기는 능력보다, 무엇을 앱에서 구현하고 무엇을 백엔드에 맡길지 정확하게 설계하는 능력이 더 중요하다는 것입니다.

Chapter09

챕터 09. 모바일 앱 개발 전체 과정 정리와 회고

핵심 주제: 웹 서비스를 모바일 앱으로 확장하기

1. 강의의 목적

이번 챕터는 모바일 앱 개발 과정에서 배운 내용을 전체적으로 정리하고, 처음 계획했던 것과 실제로 구현한 결과를 비교하는 회고 시간입니다.

이번 과정의 가장 중요한 목표는 기존 웹 서비스를 처음부터 다시 개발하는 것이 아니라, 기존 백엔드는 그대로 활용하면서 모바일 앱이라는 새로운 사용자 접점을 만드는 것이었습니다.

전체 학습 내용은 다음 다섯 가지로 정리할 수 있습니다.

1. 모바일 앱 개발 방식 선택
2. Expo의 작동 구조 이해
3. 모바일 앱과 백엔드 연결
4. 코딩 에이전트를 활용한 화면 수정
5. 스마트폰 카메라와 OCR 기능 추가

. 모바일 앱 개발 방식 선택

모바일 앱을 만드는 방법에는 여러 가지가 있습니다.

개발 방식	특징	이번 강의에서의 판단
PWA	웹사이트를 앱처럼 설치	기기 기능과 스토어 배포에 제약
웹 래핑	기존 웹 화면을 앱 안에 넣음	기존 웹 활용은 쉽지만 별도 네이티브 환경 필요
네이티브	iOS와 Android에 맞춰 각각 개발	개발 부담이 큼
React Native + Expo	JavaScript 기반으로 모바일 앱 개발	이번 프로젝트에 적합

이번 강의의 조건은 다음과 같았습니다.

- Windows 노트북 중심의 개발 환경
- 비개발자도 코딩 에이전트의 도움을 받아 개발
- 이미 작동하는 백엔드가 존재
- iOS와 Android 모두 고려

이 조건에서는 React Native와 Expo를 활용하는 방법이 적합했습니다.

특히 Expo EAS를 이용하면 iOS 빌드에 필요한 Mac 환경을 직접 준비하지 않고 클라우드에서 앱 파일을 만들 수 있다는 장점이 있습니다.

3. Expo의 구조와 작동 원리

Expo는 React Native 앱을 개발하고 실행하고 배포하는 데 필요한 도구들을 제공합니다.

핵심 구성은 다음과 같습니다.

구성 요소	역할
Expo CLI	개발 서버 실행 및 프로젝트 관리
Metro	JavaScript 코드 번들 생성
Expo SDK	카메라, 사진첩 등 기기 기능 제공
Expo Router	화면 이동과 주소 관리
Expo Go	개발 중 스마트폰에서 앱 실행
EAS Build	클라우드에서 최종 앱 파일 빌드
EAS Update	지원되는 JS·이미지 등 변경사항 업데이트

반드시 구분할 세 가지

- Expo Go: 개발 중 앱을 스마트폰에서 실행하는 도구
- 개발용 빌드: Expo Go에 없는 네이티브 기능을 포함해 테스트하는 앱
- EAS Build: 실제 설치·배포할 앱 파일을 클라우드에서 만드는 서비스

Expo Go에 미리 포함되지 않은 네이티브 기능을 사용하려면 개발용 빌드가 필요합니다.

또한 프로젝트의 Expo SDK와 Expo Go가 지원하는 SDK 버전이 맞아야 정상적으로 실행됩니다.

4. 앱과 백엔드의 역할 분리

이번 강의 전체를 관통하는 가장 중요한 설계 원칙입니다.

모바일 서비스의 전체 구조

웹 화면

기존 사용자 접점

모바일 앱

새로운 사용자 접점

공통 백엔드

인증 · API · AI 판단 · OCR · 약속 생성

Supabase 및 데이터베이스

사용자 · 대화 · 캐릭터 · 약속 데이터

앱은 화면과 스마트폰의 기기 기능을 담당합니다.

백엔드는 데이터 저장, 사용자 인증, AI 판단, OCR, 약속 생성 등을 담당합니다.

따라서 웹과 앱은 서로 다른 화면을 가지고 있지만 동일한 백엔드 API를 사용할 수 있습니다.

보안에서 반드시 기억할 점

`EXPO_PUBLIC_`으로 시작하는 환경변수는 앱 파일에 포함되므로 공개 가능한 정보만 넣어야 합니다.

- 백엔드 주소: 앱에 포함 가능
- Supabase publishable key: 공개 가능한 키
- Supabase secret key: 백엔드에만 보관
- LLM API 키: 백엔드에만 보관
- 데이터베이스 비밀번호: 백엔드에만 보관

사용자에게 배포되는 앱에 비밀정보를 넣지 않는 것이 기본 원칙입니다.

5. 구조를 정하고 코딩 에이전트에게 맡기기

이번 과정에서 코딩 에이전트와 협업하는 방법도 배웠습니다.

단순히 "이런 기능을 만들어 줘"라고 요청하는 것이 아니라, 먼저 기능의 구조를 정리했습니다.

예를 들어 기억과 약속 메모 기능을 만들 때 다음을 먼저 결정했습니다.

1. 답변 아래에 어떤 정보를 보여 줄 것인가?
2. 해당 정보는 어떤 API에서 오는가?
3. 어떤 파일을 수정해야 하는가?
4. 화면에는 어떤 모양으로 표시할 것인가?

이후 코딩 에이전트에게 구현을 맡겼습니다.

개발 과정에서 유용했던 기능

- 코드를 저장하면 Expo Go 화면에 변경사항 즉시 반영
- 빨간 오류 화면에서 파일과 줄 번호 확인

- 오류 메시지를 코딩 에이전트에게 전달해 수정
- `npx tsc --noEmit`으로 타입 검사
- `npx expo-doctor`로 프로젝트 설정과 의존성 검사

이번 강의의 중요한 교훈은 코딩 에이전트가 코드를 작성하더라도 무엇을 만들지 결정하는 것은 사람의 역할이라는 점입니다.

6. 스마트폰 카메라와 OCR 기능 추가

이번 강의에서 실제로 사용자 경험을 개선한 기능입니다.

스마트폰 카메라나 사진첩에서 일정표를 가져와 글자를 읽고, 약속을 생성하도록 만들었습니다.

기능	사용 기술	역할
사진첩에서 선택	`expo-image-picker`	기존 사진 가져오기
카메라로 촬영	`expo-camera`	현장에서 사진 촬영
사진 글자 읽기	백엔드 OCR API	이미지에서 텍스트 추출
약속 생성	기존 AI 백엔드	추출한 내용을 바탕으로 약속 생성

여기서 중요한 것은 앱이 OCR 자체를 수행하지 않는다는 것입니다.

앱은 사진을 가져와 백엔드에 전달하고, OCR과 약속 생성은 백엔드가 처리합니다.

또한 OCR 결과를 바로 전송하지 않고 사용자가 확인하고 수정하도록 했습니다.

이것은 AI가 만든 결과를 사람이 검토한 뒤 실행하는 HITL(Human-in-the-Loop) 구조입니다.

7. 주관식 퀴즈: 웹 서비스에서 모바일 앱으로 확장하기

Q. 섹션 1에서 웹 서비스를 앱으로 만들 때 새로 필요한 것과 그대로 쓸 것을 나눠 적었습니다. 그 답을 다시 읽고, 오늘 실제로 새로 만든 것과 그대로 쓴 것을 비교해 보세요.

모범 답안

처음에는 웹 서비스를 모바일 앱으로 전환하기 위해 화면, 실행 환경, 기기 기능 등을 새로 만들어야 한다고 생각했습니다. 반면 AI 판단, 사용자 데이터 관리, 대화 처리 등의 백엔드 기능은 기존 것을 그대로 활용할 수 있다고 판단했습니다.

실제 개발에서도 이 방향대로 진행했습니다.

구분	새로 만든 것	그대로 사용한 것
사용자 화면	React Native 기반 모바일 화면	기존 웹 화면과 디자인·기능 참고
개발 환경	Expo 프로젝트와 Expo Go 실행 환경	기존 백엔드 개발 환경
로그인	모바일 앱의 Supabase 익명 로그인 연결	기존 Supabase 인증 및 데이터 관리 기반
대화 기능	모바일 대화 화면과 스트리밍 표시	기존 `/api/chat`
캐릭터 기능	모바일 캐릭터 선택 화면	기존 캐릭터 데이터와 API
약속 기능	모바일에서 약속 메모 표시	기존 백엔드 약속 생성 기능
사진 기능	카메라·사진첩 연동	기존 `/api/ocr` 및 OCR 처리
보안	모바일 환경변수 관리	백엔드의 비밀정보 관리

결론적으로 새롭게 만든 것은 모바일이라는 사용자 접점이고, 기존에 잘 작동하던 백엔드의 데이터와 AI 기능은 최대한 재사용했습니다.

이것이 이번 모바일 앱 개발 과정에서 얻은 가장 중요한 결과입니다.

8. 나의 프로젝트에 적용하기

이번 강의에서 배운 구조는 앞으로 진행할 여러 AI 서비스에 공통으로 적용할 수 있습니다.

프로젝트	새롭게 만들 사용자 접점	재사용할 백엔드 기능
AI 친구 에이전트	모바일 대화·카메라 화면	AI 대화·기억·약속
MSM	모바일 일기·사진·음성 입력	글 분석·웹소설 생성
Travel Creator	여행 중 사진·일정 입력	여행 일정 분석·추천
ASP 컨설팅	고객 진단·결과 조회 화면	AI 진단·보고서 생성

특히 앞으로 1인 창업을 준비하면서 기억할 점이 있습니다.

새로운 서비스를 만들 때마다 AI 엔진과 데이터베이스를 처음부터 개발할 필요는 없습니다.

공통 백엔드를 잘 설계해 두면 웹, 모바일, 고객용 포털 등 새로운 접점을 추가할 때 개발 부담을 줄일 수 있습니다.

9. 반드시 기억해야 할 핵심 8가지

1. 모바일 앱 개발 방식은 PWA, 웹 래핑, 네이티브, React Native + Expo로 구분됩니다.
2. Expo는 Windows 환경에서도 클라우드 빌드를 통해 iOS와 Android 앱 개발을 지원합니다.
3. Expo Go는 개발 테스트 도구이고, 최종 앱 파일은 EAS Build로 만듭니다.

4. 앱은 화면과 기기 기능을 담당하고 백엔드는 데이터와 AI 판단을 담당합니다.
5. 앱과 백엔드는 API와 로그인 토큰으로 연결됩니다.
6. `EXPO_PUBLIC_` 환경변수에는 공개 가능한 정보만 넣어야 합니다.
7. 카메라와 사진첩은 Expo 라이브러리로 연결하고 OCR은 백엔드에서 처리할 수 있습니다.
8. AI가 처리한 결과를 사람이 확인한 뒤 실행하는 HITL 구조가 중요합니다.

10. 오늘 강의 한 문장 요약

기존 웹 서비스의 백엔드는 그대로 활용하면서 Expo로 모바일 화면과 스마트폰 기능을 추가하고, API를 통해 두 환경을 연결하는 것이 이번 모바일 앱 개발 과정의 핵심입니다.

11. AI 서비스 기획자 관점의 최종 교훈

이번 1~9장 강의에서 배운 것은 단순한 모바일 앱 개발 기술이 아닙니다.

기존에 운영하던 AI 서비스를 새로운 플랫폼으로 확장하는 방법입니다.

앞으로 AI 서비스를 기획할 때 다음 세 가지를 먼저 생각하시면 좋겠습니다.

- 무엇을 새로 만들 것인가? 사용자 화면, 입력 방식, 기기 기능
- 무엇을 재사용할 것인가? 백엔드 API, AI 엔진, 데이터베이스
- 어떤 역할을 분리할 것인가? 사용자 인터페이스, 데이터 처리, AI 판단, 보안

이 세 가지를 명확히 구분할 수 있다면 비개발자라도 코딩 에이전트에게 훨씬 정확한 개발 지시를 내릴 수 있습니다.

최종 교훈: 좋은 AI 서비스 개발자는 모든 것을 새로 만드는 사람이 아니라, 이미 가진 기술과 기능을 연결해 더 많은 사용자에게 새로운 경험을 제공하는 사람입니다.